



數據加密

李琦

清華大學網研院



本章的內容組織



第一節 密碼學簡史

- 古典密碼
- 近代密碼
- 現代密碼

回顧歷史，瞭解
密碼學演進方向

為密碼算法的優化提
供歷史經驗

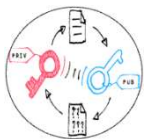


第二節 對稱密碼

- 分組密碼
- DES算法
- 流密碼

總結對稱密碼的
加密解密原理

密碼學方法為網絡空間的安全穩
定保駕護航



第三節 公鑰密碼

- 密鑰分發問題
- 公鑰密碼
- RSA算法
- 應用場景

剖析公鑰密碼的
來龍去脉



第四節 摘要與簽名

- 散列函數
- 消息認證碼
- 數字簽名

介紹校驗數據
完整性的工具方法



第五節 公鑰基礎設施PKI

- 體系結構
- 信任模型
- 安全問題
- 應用場景

明確PKI的意義及如
何構建安全的PKI

解決公鑰密碼中的
公鑰真實性問題



第六節 密碼分析技術

- 唯密文攻擊
- 已知明文攻擊
- 選擇明文攻擊
- 選擇密文攻擊
- 選擇密鑰攻擊

梳理密碼分析領域
的技術體系

對密碼算法的設計
具有理論指導意義



第1節 密碼學簡史

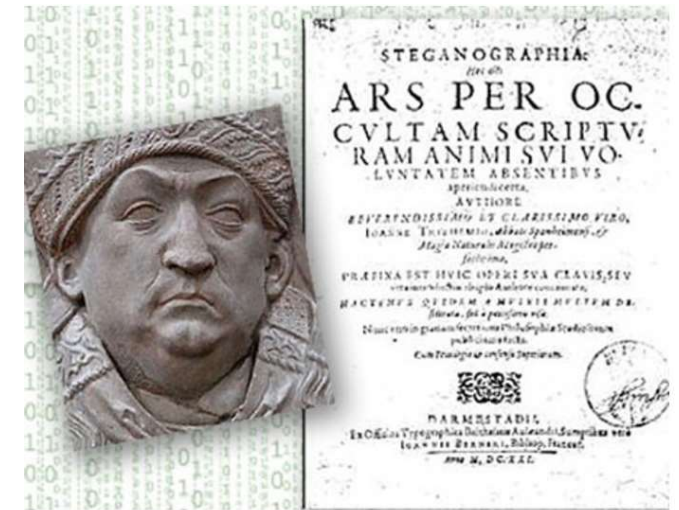
- ✓ 古典密碼
- ✓ 近代密碼
- ✓ 現代密碼





隱寫術與密碼學

- 隱寫術是一門關於信息隱藏的技巧與科學
- 隱寫術的英文叫做Steganography，來源于Histaiaeus的一本講述密碼學與隱寫術的著作《Steganographia》，書名源于希臘語，意為“隱秘書寫”

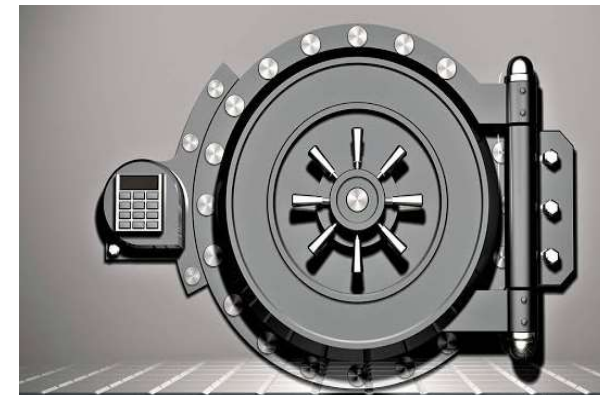


隱寫術與密碼學有何區別？



密碼學

- 密碼學 (cryptography)
- 源于希臘語kryptós “隱藏的”，和gráphein “書寫”
- 研究如何隱密地傳遞信息的學科



“密碼學是關於如何在敵人存在的環境中通訊”

——著名的密碼學者Ron Rivest (RSA作者，圖靈獎獲得者)



Ron Rivest



密碼學的基本概念

- **密碼編碼**：通過信息編碼使信息保密
- **密碼分析**：用分析方法解密信息
- **基本術語**：



- 明文(plain text), 密文(cipher text)
- 加密(encrypt, encryption), 解密(decrypt, decryption)
- 密碼算法(Algorithm), 密碼(Cipher): 用來加密和解密的數學函數

$$c=E(m), \quad m=D(c), \quad D(E(m))=m$$

- 密鑰(Key): 密碼算法中的一個變量

$$c=E_{K_e}(m), \quad m=D_{K_d}(c), \quad D_{K_d}(E_{K_e}(m))=m$$



密碼編碼

- 把明文轉換成密文的兩種操作
 - **代替**: each element in the plaintext is mapped into another element
 - **換位**: elements in the plaintext are rearranged
- 密鑰的數量
 - Symmetric, secret key, conventional encryption
 - Asymmetric, two keys, public key encryption
- 對明文加密的方式
 - Block cipher
 - Stream cipher



密碼技術的主要用途

- **數據保密—數據加密/解密**
 - 數據加密(存儲和傳輸)
- **認證技術**
 - 實體身份認證
 - 數據源發認證
- **信息完整性保護**
 - 傳輸過程中沒有被插入、篡改、重發
- **數字簽名和抗抵賴**
 - 源發抗抵賴
 - 交付抗抵賴





密碼學簡史

這一時期的密碼學更像是一門藝術，其核心的密碼編碼方法歸根結底主要有兩種，即代替和換位

古典密碼

1949年以前

近代密碼

香農發表了“保密系統的通信理論”為密碼系統建立了理論基礎，是密碼發展史上的第一次飛躍，密碼學邁入近代密碼時期



密碼專家迪菲和赫爾曼發表文章“密碼學的新方向”提出的公鑰密碼建立在數學問題難解性問題之上，是密碼學的一次真正的革命

現代密碼

1976年~至今





古典密碼

- 古典密碼
 - 代替密碼 (Substitution Cipher)
 - 換位密碼 (Transposition Cipher)
 - 代替密碼與換位密碼的組合
 - 密碼體制的安全性在于保持**算法**本身的保密性
- 缺陷（受限算法的缺陷）
 - 不適合大規模使用
 - 不適合較大的或者人員變動較大的組織
 - 用戶無法瞭解算法的安全性



Caesar密碼

凱撒(Caesar)密碼是公元前一世紀在高盧戰爭時被使用的，它是將英文字母向前移動k位

明文	A	B	C	D	E	F	G	H	I	J	K	L	M
密文	F	G	H	I	J	K	L	M	N	O	P	Q	R
明文	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
密文	S	T	U	V	W	X	Y	Z	A	B	C	D	E

凱撒密碼明密文對應表

明文: i went to school this morning

密文: n vjsz zu yhmuuq zmny ruxsnsl

Caesar 密碼 : $c = (m + 5) \text{ MOD } 26$



猪圈密碼

猪圈密碼是一種以格子為基礎的簡單替代式密碼，格子如下：

A	B	C
D	E	F
G	H	I

J	K	L
M	N	O
P	Q	R

	S	
T	X	U
	V	

	W	
X	Y	Z

示例：

密文為： □ □ √ ⊥ ⊥ □ < □ <

明文為： H O W A R E Y O U



縱行換位密碼

- 換位密碼是一種早期的加密方法，明文的字母保持相同，但順序被亂
- 廣泛應用于美國南北戰爭

輸入方向 →

C	A	N	Y
O	U	U	N
D	E	R	S
T	A	N	D

輸出方向 ↓

明文：Can you understand

密文：
C
o
d
t
a
u
e
a
n
u
r
n
y
n
s
d



戚繼光 反切碼

反切碼源自反切拼音注音方法，是用兩個字為另一個字注音，取上字的聲母和下字的韻母，“切”出另外一個字的讀音



詩篇1 聲母

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15					
l	b	q	q	d	b	t	zh	r	sh	y	m	y	ch	x	d	zh	y	j	zh
柳	邊	求	氣	低	,	波	他	爭	日	時	,	鶯	蒙	語	出	喜	,	打	掌
																		與	君
																			知

詩篇2 韻母

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
un	ua	iang	iu	an	ai	ia	in	uan	e	u	in	ei	u	eng	uang	ui	ao	in
春	花	香	,	秋	山	開	,	嘉	賓	歡	歌	須	金	杯	,	孤	燈	光
20		21	22	23		24	25	26		27	28	29	30	31	32	33	34	35
ang		i	ong	iao		uo	i	iao		i	eng	ui	u	ian	i	ei	ai	e
缸	。	之	東	郊	,	過	西	橋	,	鷄	聲	催	初	天	,	奇	梅	歪
																		遮
																		溝
																		。



戚繼光 反切碼

反切碼源自反切拼音注音方法，是用兩個字為另一個字注音，取上字的聲母和下字的韻母，“切”出另外一個字的讀音

舉例：

密文：

5-25-2

5
d
低

+

25
i
西

→

di

→

明文： 敵

詩篇1
聲母

詩篇2
韻母

聲調
2聲

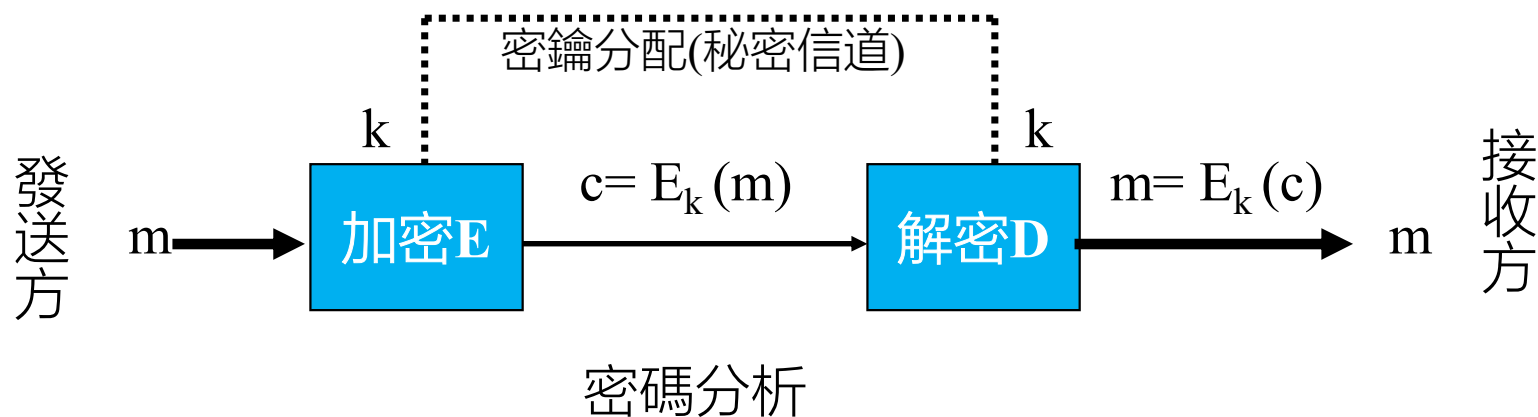




近代密碼

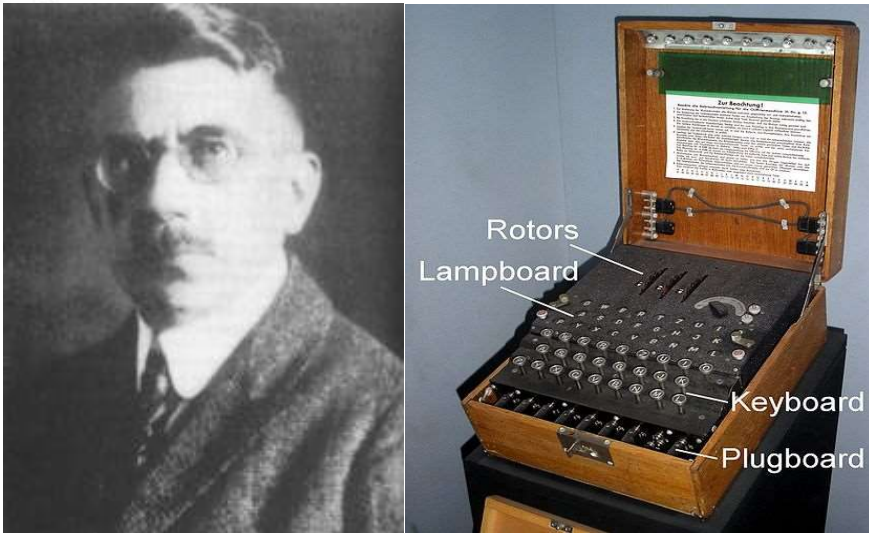
1949年，香農發表的“Communication Theory of Secrecy Systems”為密碼系統建立了理論基礎，是密碼發展史上的**第一次飛躍**，使密碼技術由藝術變成了科學

- 將**算法**和**密鑰**分開
- 密碼算法可以公開，密鑰保密
- 密碼系統的安全性在於保持密鑰的保密性





ENIGMA機



Arthur Scherbius與轉輪密碼機ENIGMA

- 轉輪密碼機ENIGMA，由Arthur Scherbius于1919年發明，1926年裝備德軍



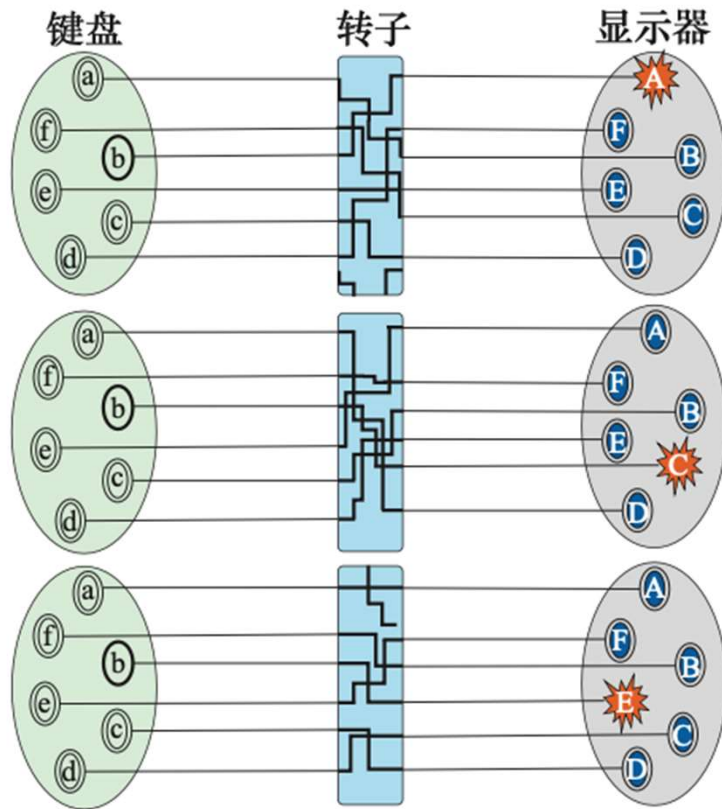
TYPEX打字密碼機

- 英國的TYPEX打字密碼機，是德國3輪ENIGMA的改進型密碼機，它在英國通信中廣泛使用



ENIGMA機工作原理

連續鍵入三個b:

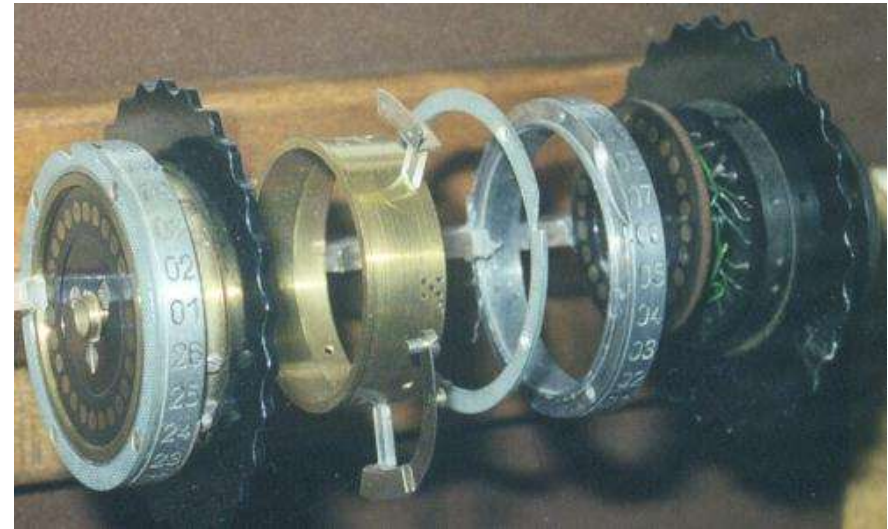


a	→	B
b	→	A
c	→	D
d	→	F
e	→	E
f	→	C

a	→	D
b	→	C
c	→	B
d	→	E
e	→	A
f	→	F

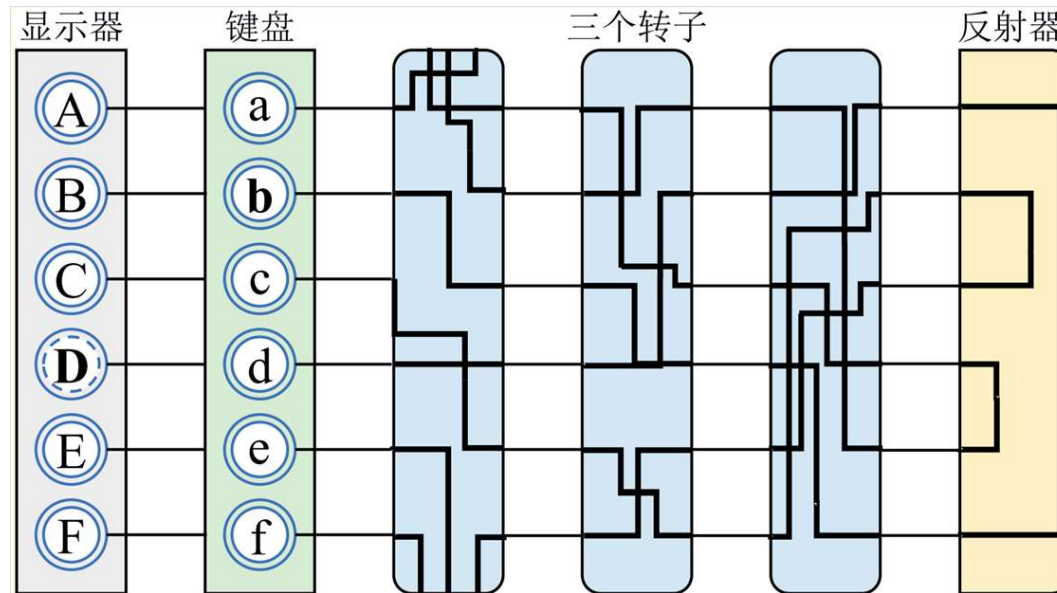
a	→	A
b	→	E
c	→	D
d	→	C
e	→	F
f	→	B

實物圖:





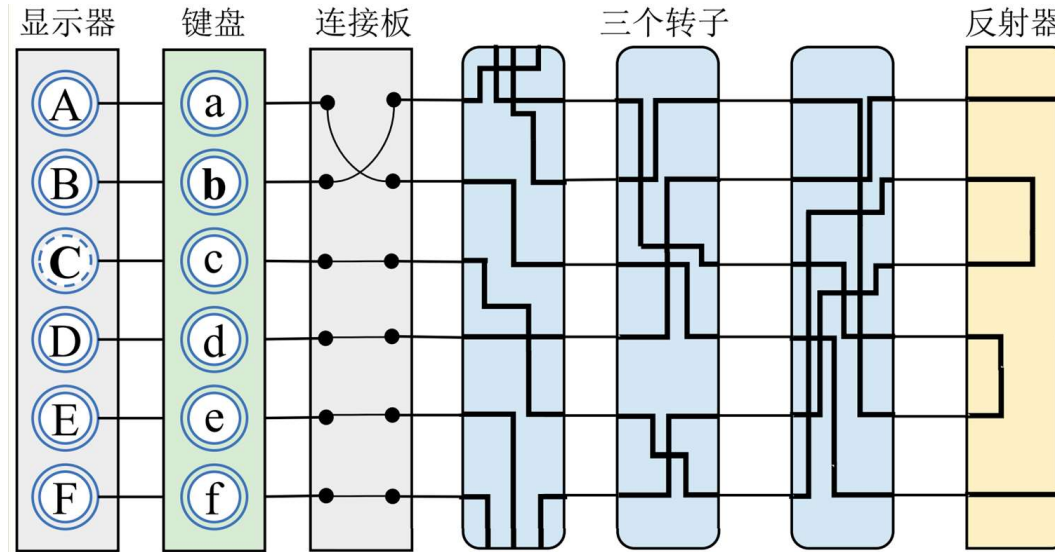
ENIGMA機工作原理



- 第一個轉子轉一圈，第二個轉子轉一位
- 在三個轉子的一端加上了一個反射器，反射器可以使譯碼的過程和編碼的過程完全一樣



ENIGMA機工作原理



- 轉子的初始設置，轉子之間的相互位置，以及連接板連線的狀況組成了所有可能的密鑰
- 三個轉子不同的方向組成了 $26*25*26=16900$ 種不同可能性
- 三個轉子間不同的相對位置為6種可能性
- 連接板上兩兩交換6對字母的可能性為 $C(26,6)*C(20,6)*6!/64 = 100391791500$
- 共有 $16900*6*100391791500$ ，約為一億億種可能性



ENIGMA工作過程

- **根據密碼本取得當日密鑰**
- **首先發送一個新的密鑰**
 - 隨機地選擇三個字母，比如說PGH
 - 把PGH在鍵盤上連打兩遍，加密為比如說KIVBJE（注意兩次PGH被加密為不同的形式）
 - 把KIVBJE放在在電文的最前面
- **重新調整三個轉子的初始方向到PGH**
- **正式對明文加密**



ENIGMA破解



Marian Adam Rejewski



Jerzy Witold Różycki



Henryk Zygalski

- 20世紀30年代，馬裏安·雷耶夫斯基、杰爾茲·羅佐基與亨裏克·佐加爾斯基等人一同進行了對德國Enigma密碼的破譯工作，并稱為密碼研究領域的“波蘭三杰”
- 2000年7月17日，波蘭政府向雷耶夫斯基、羅佐基和佐加爾斯基追授波蘭最高勳章

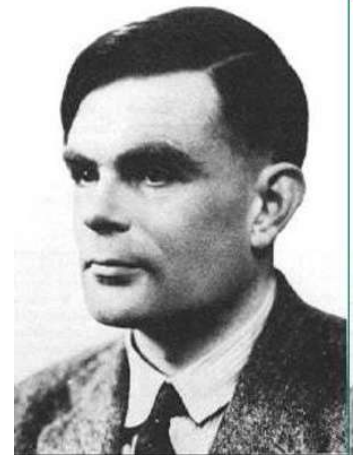


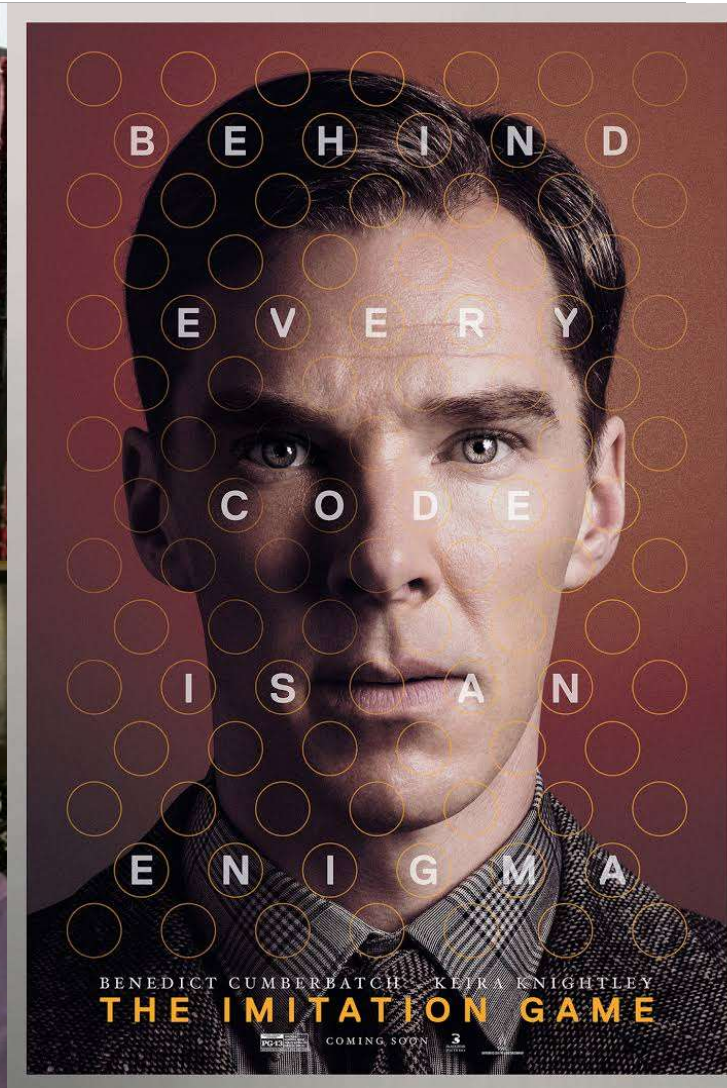
ENIGMA破解

- 阿蘭·麥席森·圖靈 (1912.6.23—1954.6.7)

Alan Mathison Turing

- 英國數學家、邏輯學家，他被視為計算機之父
- 1931年圖靈進入劍橋大學國王學院，畢業後到美國普林斯頓大學攻讀博士學位，二戰爆發後回到劍橋，後曾協助軍方破解德國的著名密碼系統Enigma，幫助盟軍取得了二戰的勝利







現代密碼

- 對稱密鑰密碼系統的缺陷
 - 密鑰必須經過安全的信道分配
 - 無法用于數字簽名
 - 密鑰管理複雜 $O(n^2)$

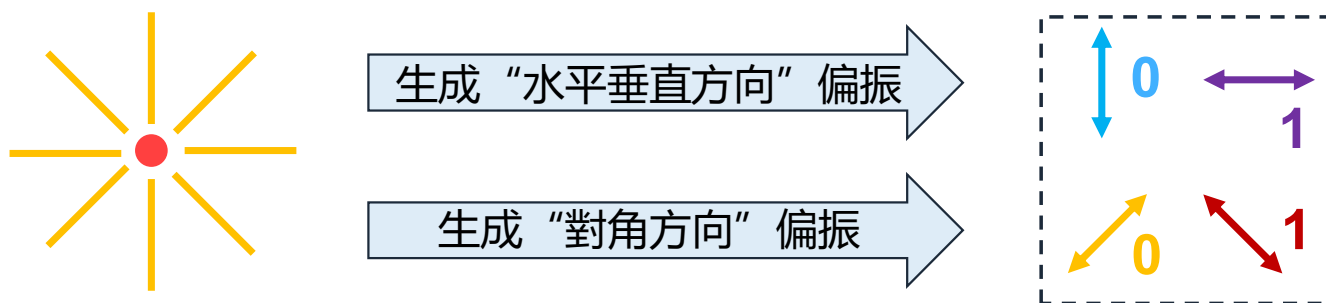


- 1976 年，Diffie和Hellman發表了《*New Direction in Cryptography*》提出了“公鑰密碼”的概念，獲得2015年圖靈獎
- 使用**兩個**密鑰，對於密鑰分配、數字簽名、認證等有深遠影響
- 基于**數學函數**而不是代替和換位，密碼學歷史上唯一的一次真正的革命，實現了密碼學發展史上的**第二次飛躍**

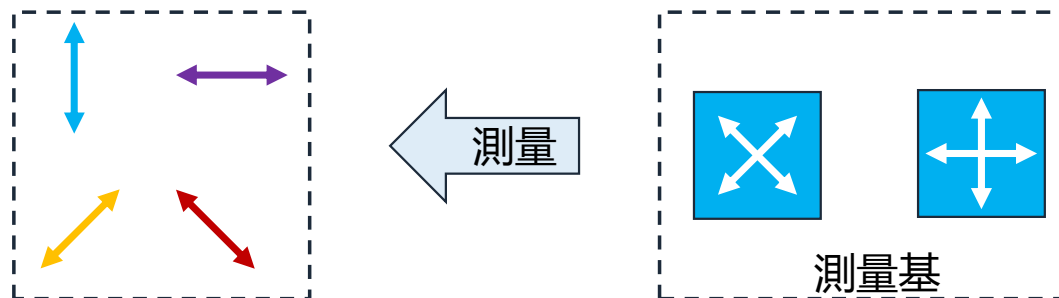


量子密碼

- 量子密碼體系采用量子態作為信息載體，經由量子通道在合法的用戶之間傳送密鑰，其安全性由量子力學原理所保證
- 光的偏振：



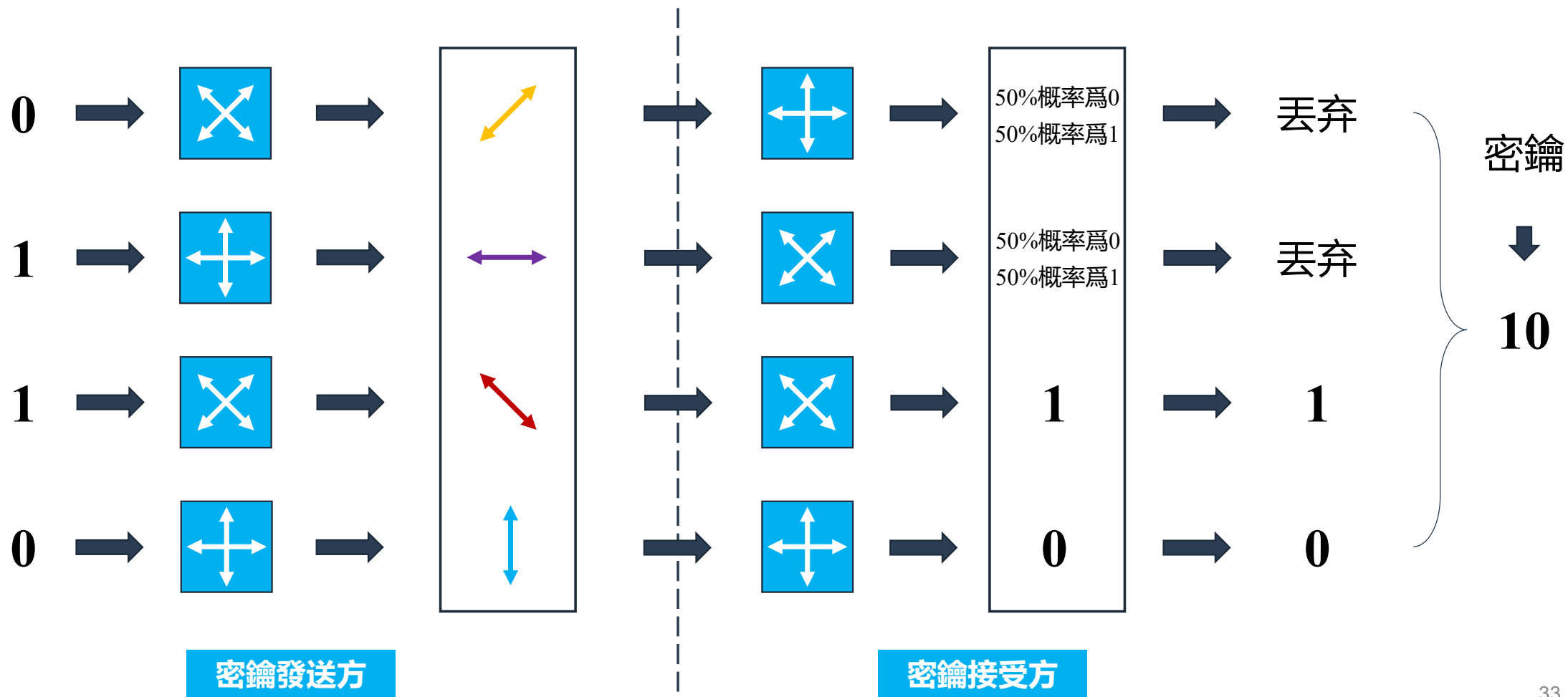
- 偏振方向測量：





量子密碼

量子密鑰生成方法：



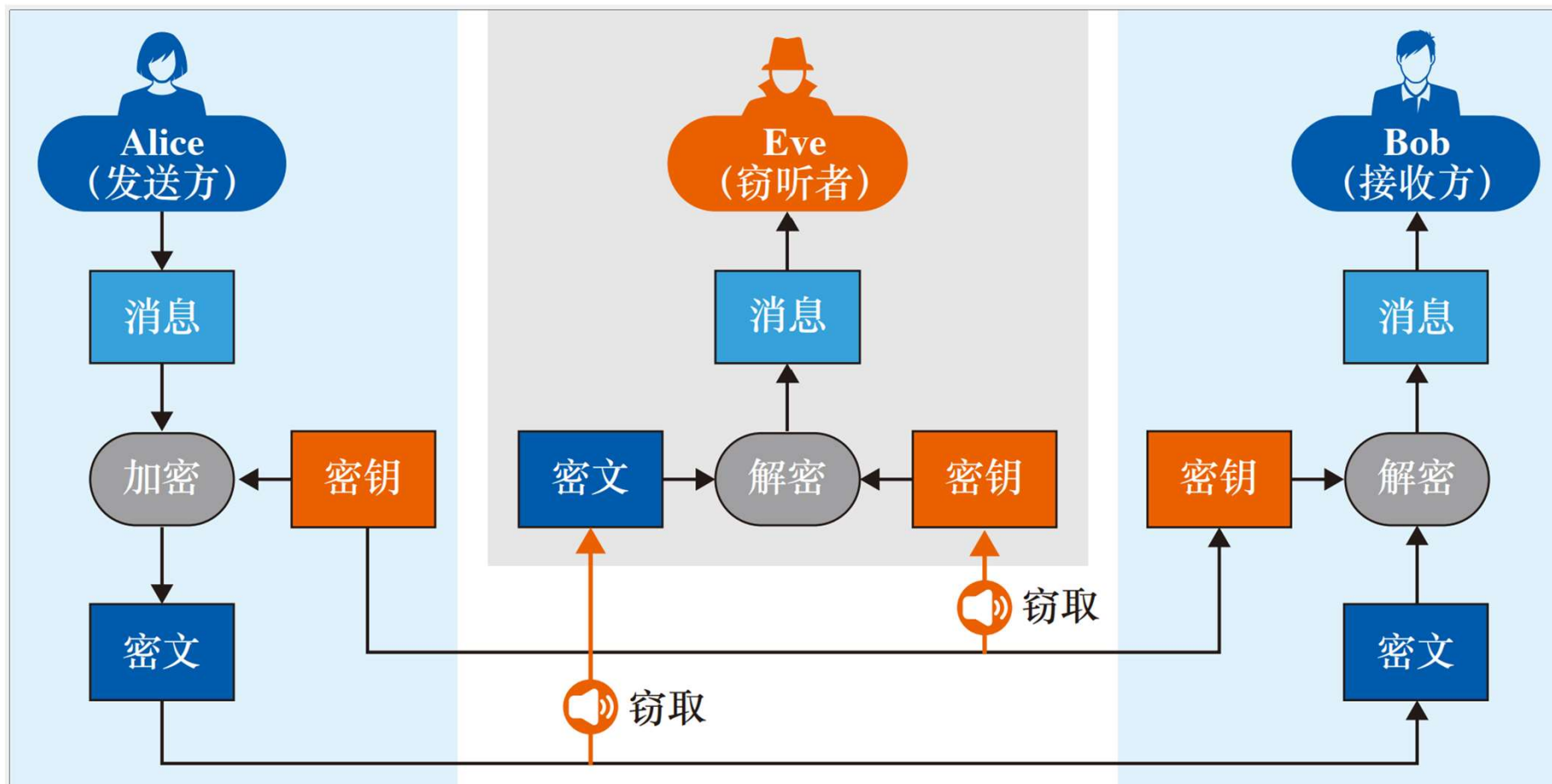


第3節 公鑰密碼

- ✓ 密鑰分發問題
- ✓ 公鑰密碼
- ✓ RSA算法
- ✓ 實際應用



密鑰分發問題





解決密鑰分發問題

- **通過事先共享密鑰來解決**

在加密通信以前，事先用安全的方式將密鑰交給接收方

- **通過密鑰分配中心來解決**

當參與加密通信的人過多，可以使用密鑰分配中心(Key Distribution Center, KDC)。當需要進行加密通信時，密鑰分配中心會生成一個通信密鑰，每個人只需和KDC事先共享密鑰即可

- **通過Diffie-Hellman密鑰交換來解決**

進行加密通信的雙方僅需要交換一些信息，該信息即使被竊聽也無法對算法進行解密，而通信雙方則可以通過該信息各自生成相同的密鑰

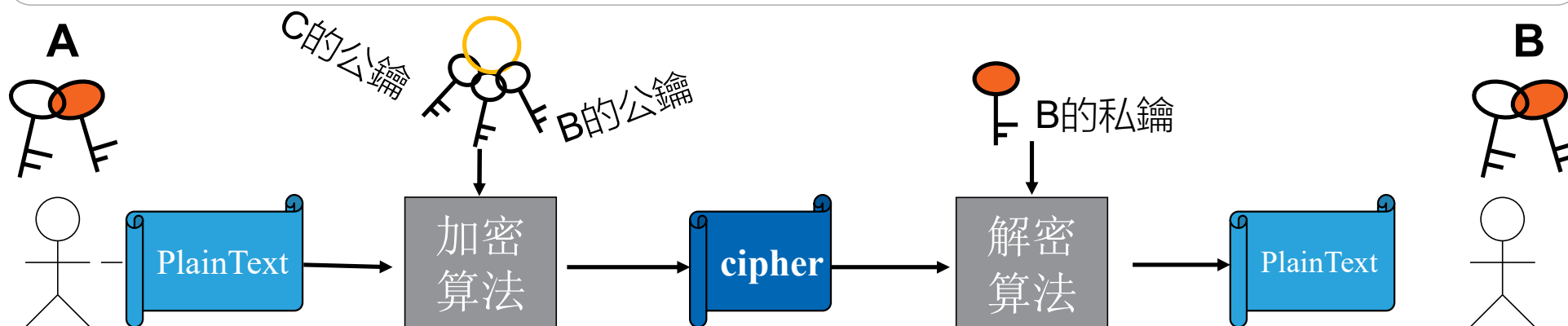
- **通過公鑰密碼來解決**

由于加解密使用不同密鑰，故無需進行密鑰分發



公鑰密碼系統的加密原理

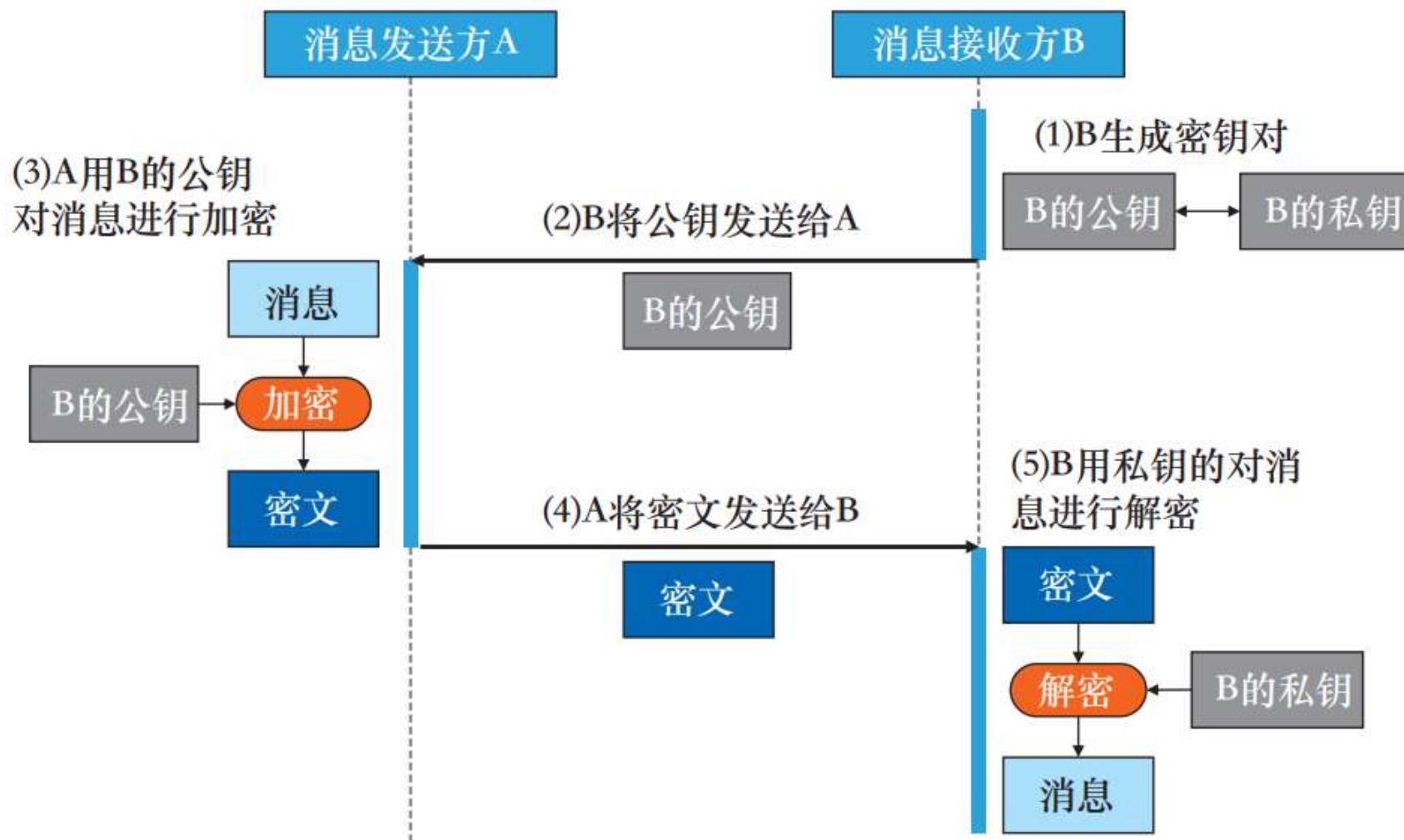
- 每個通信實體有一對密鑰(公鑰, 私鑰)
 - 公鑰公開, 用于加密和驗證簽名, 私鑰保密, 用作解密和簽名
- A 向 B 發送消息, 用 B 的公鑰加密
- B 收到密文後, 用自己的私鑰解密



- 任何人向 B 發送信息都可以使用同一個密鑰 (B 的公鑰) 加密
- 沒有其他人可以得到 B 的私鑰, 所以只有 B 可以解密

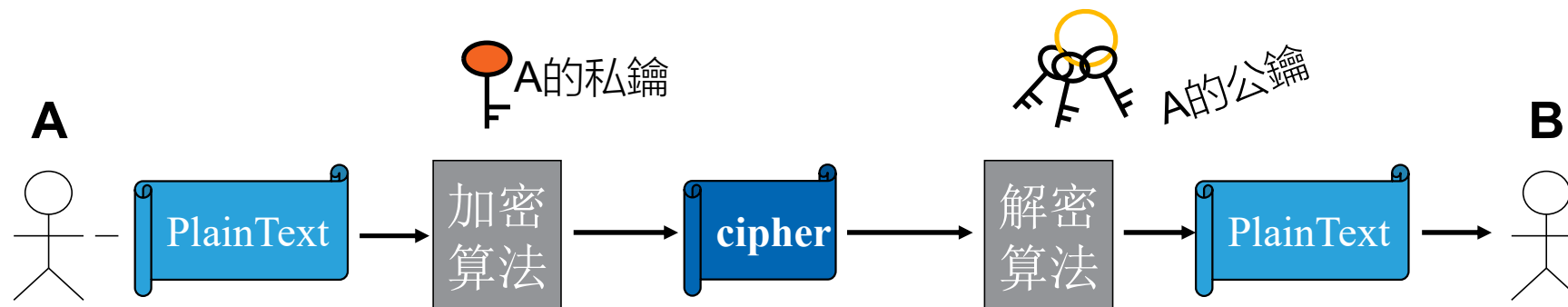


公鑰密碼系統的加密原理





公鑰密碼系統的簽名原理



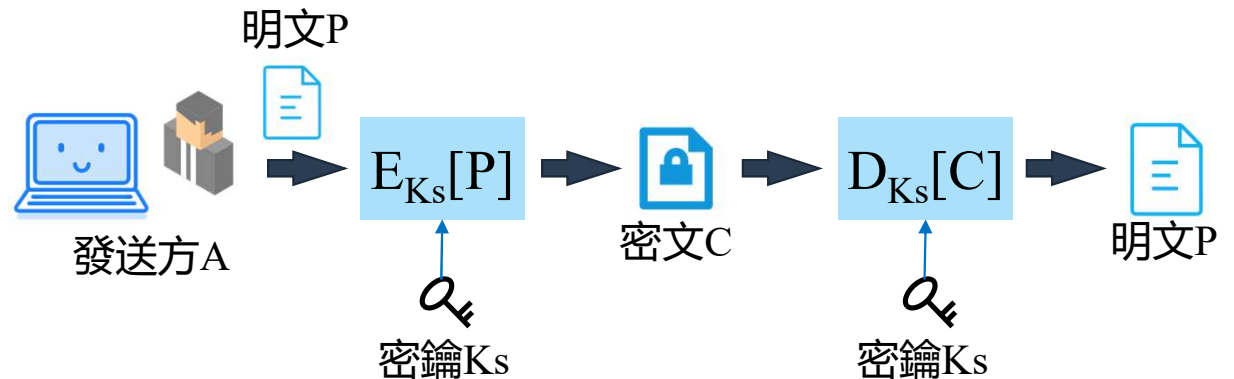
- A 向 B 發送消息，用 A 的私鑰加密 (簽名)
- B 收到密文後，用 A 的公鑰解密 (驗證)



公鑰密碼算法的表示

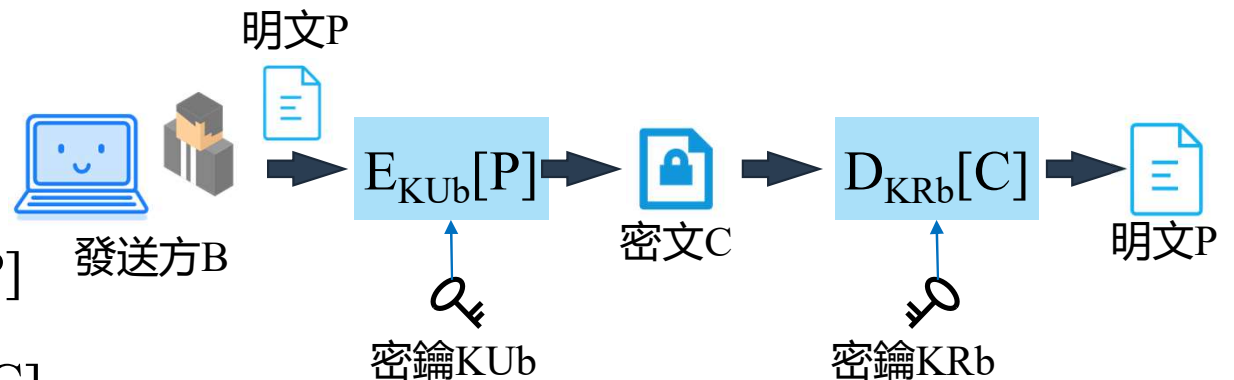
- 對稱密鑰密碼

- 密鑰：會話密鑰(K_s)
- 加密函數： $E_{K_s}[P]$
- 對密文 C ，解密函數： $P=D_{K_s}[C]$



- 公開密鑰

- (KU_a, KR_a)
- 加密/簽名： $C=E_{KU_b}[P], E_{KR_a}[P]$
- 解密/驗證： $P=D_{KR_b}[C], D_{KU_a}[C]$





對公開密鑰密碼算法的要求

- 參與方B容易產生密鑰對(KUb, KRb)

- 已知KUb, A的加密操作是容易的

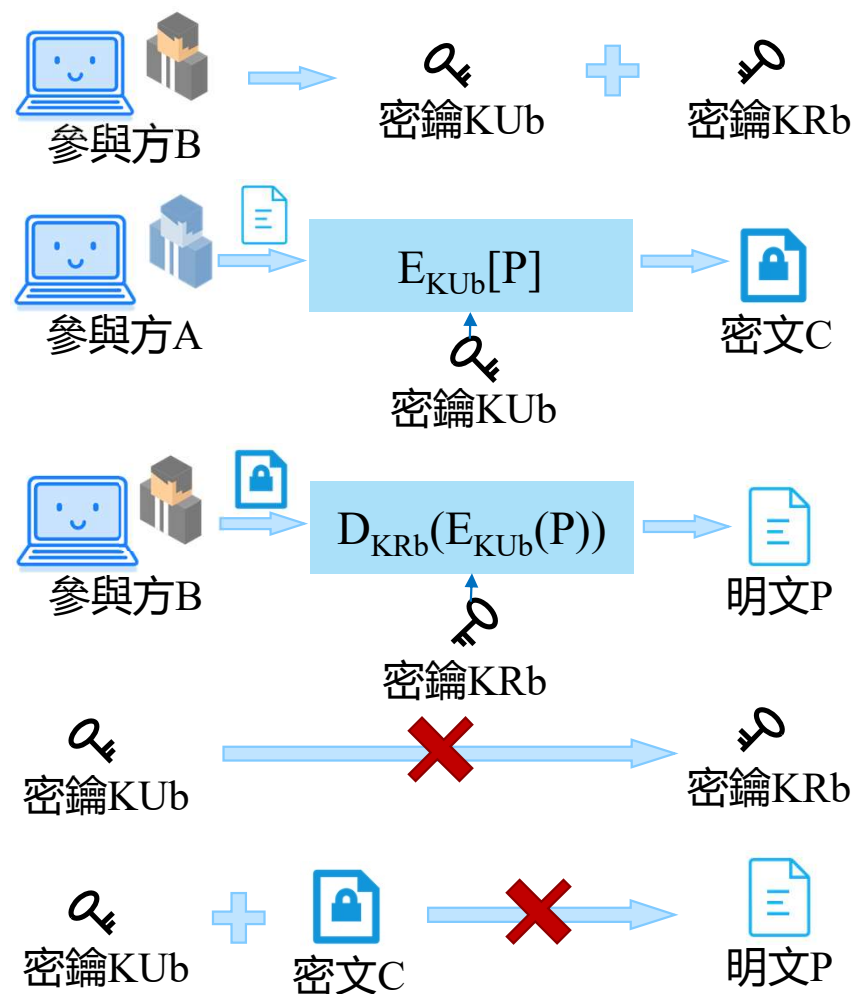
$$C = E_{KUb}(P)$$

- 已知KRb, B的解密操作是容易的

$$P = D_{KRb}(C) = D_{KRb}(E_{KUb}(P))$$

- 已知KUb, 求KRb是計算上不可行的

- 已知KUb和C, 欲恢復P是計算上不可行的





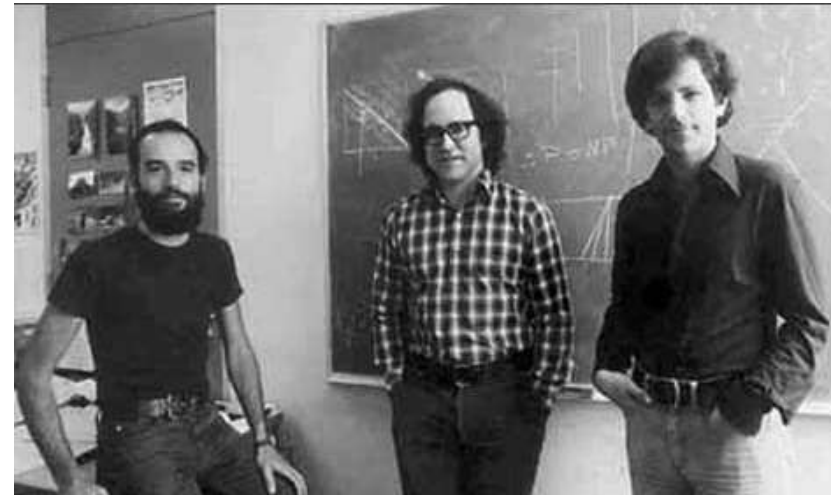
對公鑰密碼算法的誤解

- 公開密鑰算法比對稱密鑰密碼算法更安全
 - 任何一種算法都依賴于密鑰長度、破譯密碼的工作量，從抗分析角度，**沒有一方更優越**
- 公開密鑰算法使對稱密鑰成爲過時的技術
 - 公開密鑰很慢，只能用在密鑰管理和數字簽名，對稱密鑰密碼算法將**長期存在**
- 使用公開密鑰加密，密鑰分配非常簡單
 - 事實上的密鑰分配既不簡單，也不有效



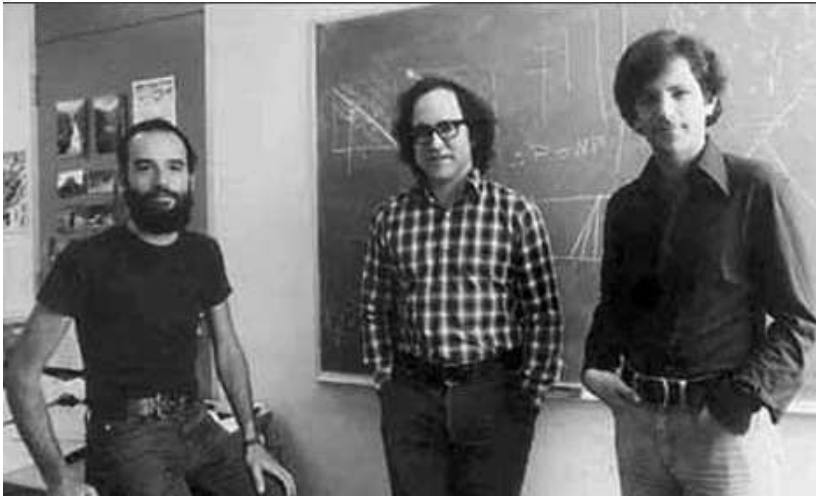
RSA算法簡介

- Ron Rivest, Adi Shamir , Leonard Adleman
——獲得2002年圖靈獎
- RSA的安全性基于**大數分解**的難度
- RSA在美國申請了專利(已經過期), 在其他國家沒有
- RSA已經成了事實上的工業標準, 在美國除外
- RSA算法既可以用于公鑰密碼也可以用于數字簽名

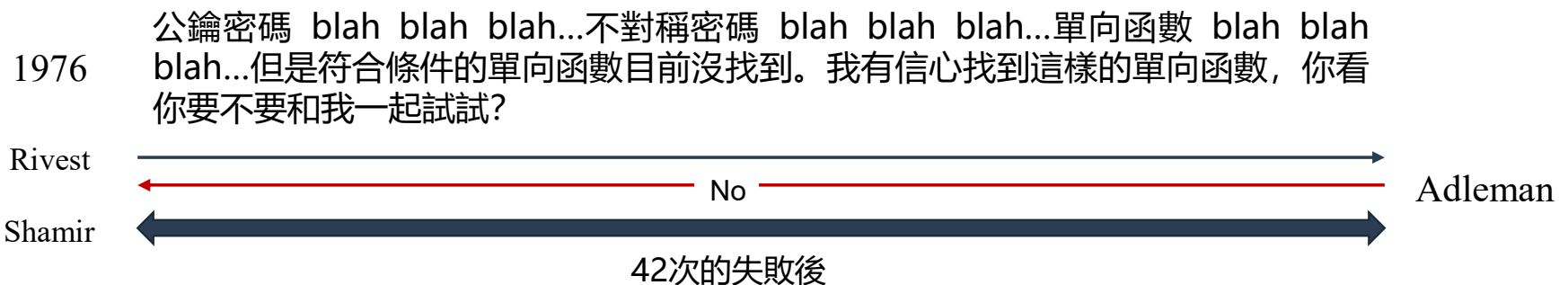




RSA算法誕生



- Ron Rivest、Adi Shamir 和 Leonard Adleman
三個人風格迥異，組成了一個技能互補的完美團隊
- 1977年，三人在MIT合作發表了一篇完整描述RSA算法的論文，RSA也正是由三人姓氏的首字母組成



作者：RL Rivest - 1978 - 被引用次数：21052 - 相关文章

A method for obtaining digital signatures and public-key cryptosystems, Published by ACM 1978



RSA論文的評審意見

A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. According to the (very short) introduction, this paper purports to present a practical implementation of Diffie and Hellman's public-key cryptosystem for applications in the electronic mail realm. If this is indeed the premise, **the paper should be rejected both for a failure to live up to it and for its irrelevance.**

I doubt that a system such as this one will ever be practical. The paper does a poor job of convincing the reader that practicality is attainable.

The introduction is only two paragraphs long, the relevant literature is not presented or cited, and there is virtually no comparison with the relevant work in the area. In summary, it looks as if this paper is a mathematical exercise **with little originality** (the authors claim that most of their ideas come from other papers), **too far from practical applicability, running against the established standards**, and **with a declared application area of dubious feasibility**. Not the kind of material our readers like to see in the journal. **Reject.**



RSA算法的性能表現

• 安全性

- 蠻力攻擊：對所有密鑰都進行嘗試
- 數學攻擊：等效于對兩個素數乘積(n)的因子分解

• 大數的因子分解是數論中的一個難題

因子分解的進展

十進制數字位數	近似比特數	得到的數據	MIPS年
100	332	1991	7
110	365	1992	75
120	398	1993	830
129	428	1994	5000
130	431	1996	500

• 速度

- 軟件實現比DES慢100倍
- 硬件實現比DES慢1000倍

8位公開密鑰的RSA

	512位	768位	1024位
加密 (秒)	0.03	0.05	0.08
解密 (秒)	0.16	0.48	0.93
簽名 (秒)	0.16	0.52	0.97
驗證 (秒)	0.02	0.07	0.08



RSA算法操作過程

• 密鑰產生

1. 取兩個大素數 p, q , **保密**

2. 計算 $n=pq$, 公開 n

3. 計算歐拉函數 $\varphi(n) = (p - 1)(q - 1)$

4. 任意取一個與 $\varphi(n)$ 互素的小整數 e , 即

$$\gcd(e, \varphi(n)) = 1; 1 < e < \varphi(n)$$

5. 尋找 d , $d < \varphi(n)$, 使得 $de \equiv 1 \pmod{\varphi(n)}$, $de = k \varphi(n) + 1$

公開 $(e, n) = (5, 119)$

將 d 保密, 丟棄 p, q

$$p=7, q=17$$

$$n=119$$

$$\varphi(n)=96$$

$$\text{取 } e=5$$

$$5d = k \times 96 + 1$$

$$\text{取 } k=4, \text{ 得到 } d=77$$



RSA算法加密/解密

- 公開密鑰: $KU=\{e, n\}$
- 秘密密鑰: $KR=d$
- 加密過程
 - 把待加密的內容分成 k 比特的分組
 $k \leq \log_2 n$, 并寫成數字, 設為 M , 則
 $C = M^e \bmod n$
- 解密過程

$$M = C^d \bmod n$$

Remember:

$$de = k\varphi(n) + 1$$

$$M^{k\varphi(n)+1} \equiv M \bmod n$$

$$\{5, 119\}$$

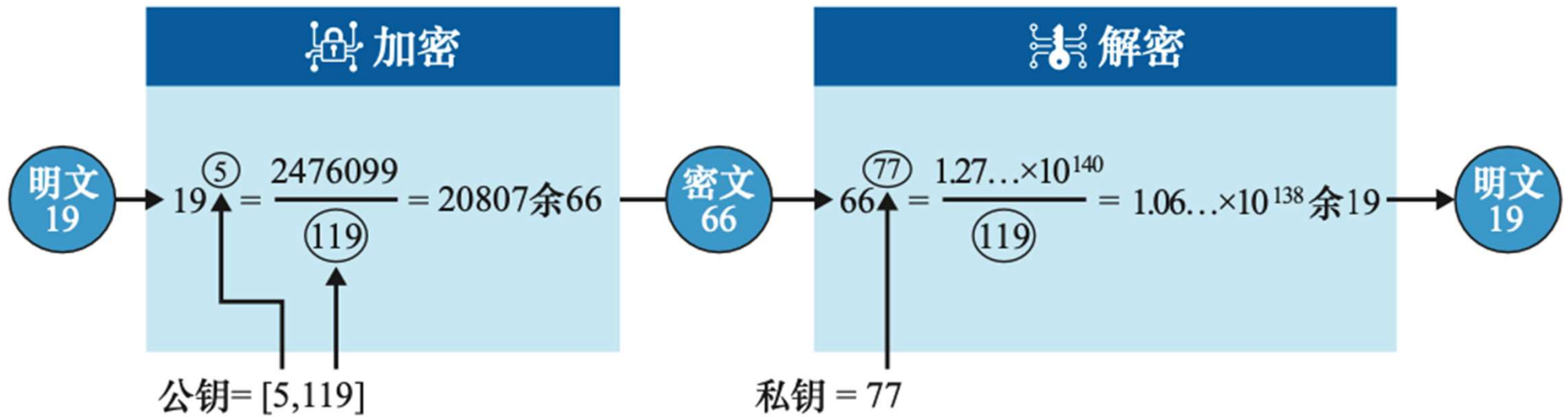
$$77$$

$$c = m^5 \bmod 119$$

$$m = c^{77} \bmod 119$$



RSA算法举例





第4節 摘要與簽名

- ✓ 散列函數
- ✓ 消息認證碼
- ✓ 數字簽名



數字簽名與摘要

- 將數據比作一個幼兒
- 數據加密即使給幼兒“穿衣服”
- 數字簽名在衣服上留下“印記”
- 摘要是孩子的“健康證明”



健康证

姓名: _____

性别: _____

年龄: _____

编号: _____

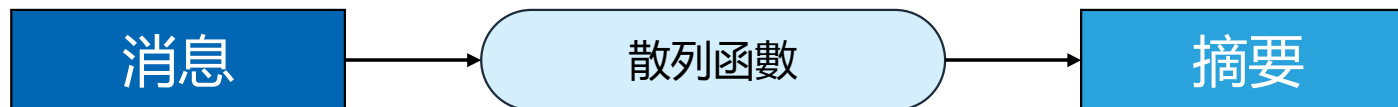
有效期至 _____ 年 _____ 月 _____ 日

照片



散列函數

- 散列函數又叫哈希 (Hash) 函數
- 把任意長度的消息壓縮為固定長度的二進制串：
散列值(hash value): $h = H(m)$
- 散列函數是進行消息認證的基本方法，主要用于消息完整性檢測和數字簽名





散列函數

- 縱向的奇偶校驗碼

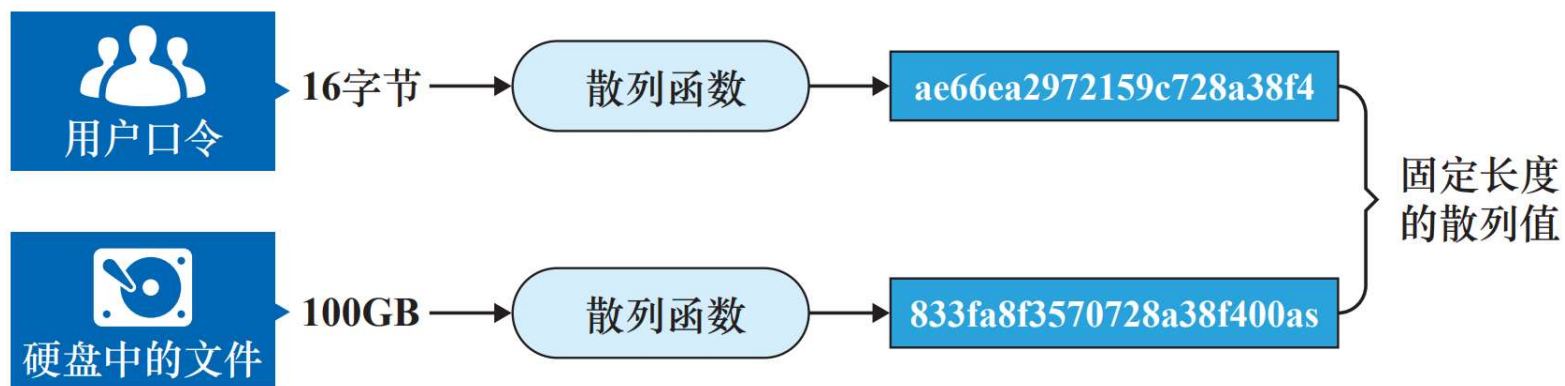
	比特1	比特2		比特n
分組1	b_{11}	b_{21}	...	b_{n1}
分組2	b_{12}	b_{22}	...	b_{n2}
	.	.	...	.
分組m	b_{1m}	b_{2m}		b_{nm}
散列碼	C_1	C_2		C_n

$$C_i = b_{i1} \oplus b_{i2} \dots \oplus b_{im}$$



散列函数的性质

1. 根据任意长度的消息计算出固定长度的散列值



2. 能够快速计算出散列值

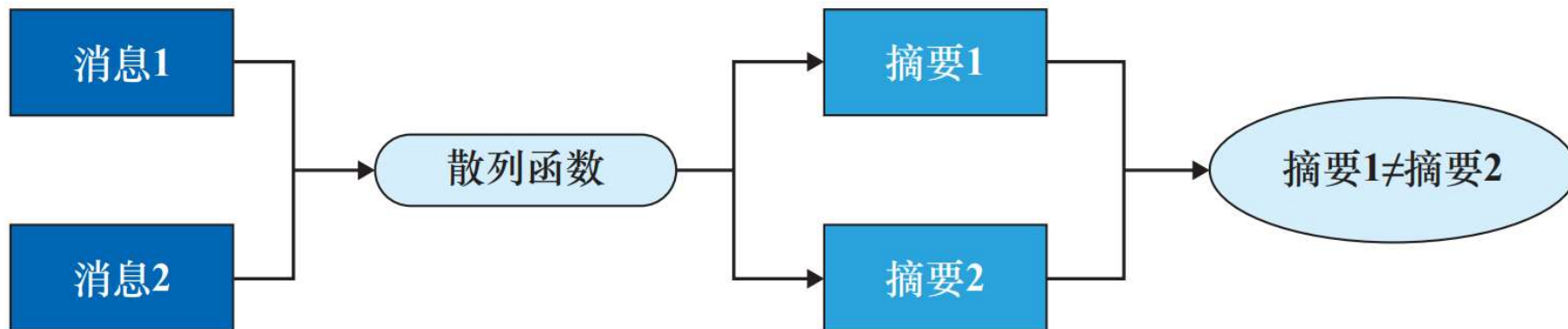
3. 单向性





散列函數的性質

4.消息不同散列值也不同





安全性：HASH 碰撞

- 2005年5月國際密碼學會議（Eurocrypt'2005）**王小雲**院士發表論文“ How to break MD5 and other hash functions”



王小雲院士

- 成功破譯**MD5**、**HAVAL-128**、**MD4**和**RIPEMD**算法
- 該次會議的總結報告寫道：“我們該怎麼辦？MD5被重創了，它即將從應用中淘汰。SHA-1仍然活著，但也看到了它的末日，現在就得開始更換SHA-1了”

How to Break MD5 and Other Hash Functions

Xiaoyun Wang and Hongbo Yu

Shandong University, Jinan 250100, China
xywang@sdu.edu.cn
yhb@mail.sdu.edu.cn

Abstract. MD5 is one of the most widely used cryptographic hash functions nowadays. It was designed in 1992 as an improvement of MD4, and its security was widely studied since then by several authors. The best known result so far was a semi free-start collision, in which the initial value of the hash function is replaced by a non-standard value, which is the result of the attack. In this paper we present a new powerful attack on MD5 which allows us to find collisions efficiently. We used this attack to find collisions of MD5 in about 15 minutes up to an hour computation time. The attack is a differential attack, which unlike most differential attacks, does not use the exclusive-or as a measure of difference, but instead uses modular integer subtraction as the measure. We call this kind of differential a *modular differential*. An application of this attack to MD4 can find a collision in less than a fraction of a second. This attack is also applicable to other hash functions, such as RIPEMD and HAVAL.

1 Introduction

People know that digital signatures are very important in information security. The security of digital signatures depends on the cryptographic strength of the underlying hash functions. Hash functions also have many other applications in cryptography such as data integrity, group signature, e-cash and many other cryptographic protocols. The use of hash functions in these applications not only ensure the security, but also greatly improve the efficiency. Nowadays, there are two widely used hash functions – MD5 [18] and SHA-1 [12].

MD5 is a hash function designed by Ron Rivest as a strengthened version of MD4 [17]. Since its publication, some weaknesses has been found. In 1993, B. den Boer and A. Bosselaers [3] found a kind of pseudo-collision for MD5 which consists of the same message with two different sets of initial values. This attack discloses the weak avalanche in the most significant bit for all the chaining variables in MD5. In the rump session of Eurocrypt'96, H. Dobbertin [8] presented a semi free-start collision which consists of two different 512-bit messages with a chosen initial value IV'_0 .

$a_0 = 0x12ac2375$, $b_0 = 0x3b341042$, $c_0 = 0x5f62b97c$, $d_0 = 0x4ba763ed$

A general description of this attack was published in [9].

R. Cramer (Ed.): EUROCRYPT 2005, LNCS 3494, pp. 19–35 2005.
© International Association for Cryptologic Research 2005



SHA1碰撞的理論計算上界

- 2005年2月7日，美國國家標準技術研究院NIST對外宣稱：SHA-1還沒有被攻破，並且也沒有足夠的理由懷疑它會很快被攻破
- **2005年8月，王小雲院士再度令世界密碼學界大跌眼鏡——SHA-1也被她攻破了！王教授提出了複雜度為 2^{69} 的理論攻擊**
- Wang, X., Yin, Y.L., Yu, H. (2005). **Finding Collisions in the Full SHA-1**. In: Shoup, V. (eds) *Advances in Cryptology – CRYPTO 2005*. CRYPTO 2005. Lecture Notes in Computer Science, vol 3621. Springer, Berlin, Heidelberg

Finding Collisions in the Full SHA-1

Xiaoyun Wang^{1,*}, Yiqun Lisa Yin², and Hongbo Yu³

¹ Shandong University, Jinan 250100, China
xywang@sdu.edu.cn

² Independent Security Consultant, Greenwich CT, US
yyin@princeton.edu

³ Shandong University, Jinan 250100, China
yhb@mail.sdu.edu.cn

Abstract. In this paper, we present new collision search attacks on the hash function SHA-1. We show that collisions of SHA-1 can be found with complexity less than 2^{69} hash operations. This is the first attack on the full 80-step SHA-1 with complexity less than the 2^{80} theoretical bound.

Keywords: Hash functions, collision search attacks, SHA-1, SHA-0.

1 Introduction

The hash function SHA-1 was issued by NIST in 1995 as a Federal Information Processing Standard [5]. Since its publication, SHA-1 has been adopted by many government and industry security standards, in particular standards on digital signatures for which a collision-resistant hash function is required. In addition to its usage in digital signatures, SHA-1 has also been deployed as an important component in various cryptographic schemes and protocols, such as user authentication, key agreement, and pseudorandom number generation. Consequently, SHA-1 has been widely implemented in almost all commercial security systems and products.

In this paper, we present new collision search attacks on SHA-1. We introduce a set of strategies and corresponding techniques that can be used to remove some major obstacles in collision search for SHA-1. Firstly, we look for a near-collision differential path which has low Hamming weight in the “disturbance vector” where each 1-bit represents a 6-step local collision. Secondly, we suitably adjust the differential path in the first round to another possible differential path so as to avoid impossible consecutive local collisions and truncated local collisions. Thirdly, we transform two one-block near-collision differential paths into a two-block collision differential path with twice the search complexity. We show that, by combining these techniques, collisions of SHA-1 can be found with complexity less than 2^{69} hash operations. This is the first attack on the full 80-step SHA-1 with complexity less than the 2^{80} theoretical bound.

* Supported by the National Natural Science Foundation of China (NSFC Grant No.90304009) and Program for New Century Excellent Talents in University.

V. Shoup (Ed.): *Crypto 2005*, LNCS 3621, pp. 17–36, 2005.
© International Association for Cryptologic Research 2005



安全性：HASH碰撞

2017年2月23號，Google在其安全博客上公布了其找到了世界首例的SHA1碰撞，標誌著SHA1不再安全了

Collision Attack: Two Different Documents, But Same SHA-1 Hash Fingerprint

SHattered

The first concrete collision attack against SHA-1
<https://shattered.io>

CWI

Google

Marc Stevens
Pierre Karpman

Elie Bursztein
Ange Albertini
Yarik Markov

SHattered

The first concrete collision attack against SHA-1
<https://shattered.io>

CWI

Google

Marc Stevens
Pierre Karpman

Elie Bursztein
Ange Albertini
Yarik Markov

```
shasum *.pdf
38762cf7f55934b34d179ae6a4c80cadccbb7f0a 1.pdf
38762cf7f55934b34d179ae6a4c80cadccbb7f0a 2.pdf
/tmp/sha1
sha256sum *.pdf
2bb787a73e37352f92383abe7e2902936d1059ad9f1ba6daaa9c1e58ee6970d0 1.pdf
```

0.64G 8-11h

58



哈希算法实例

算法名称	输出长度 (位)	简述
MD4	128	由美国密码学家 Rivest 于 1990 年针对 32 位操作系统所设计的哈希算法，虽然于 1996 年被破解，但对后来的 MD5，SHA 系列和 RIPEMD 算法产生了深远影响
MD5	128	1991 年由 Rivest 对 MD4 算法改进而来，它在 MD4 的基础上增加了“安全带”的概念。2004 年其强抗碰撞性被王小云院士团队攻破
SHA-1	160	由 NIST（美国国家标准技术研究所）MD5 的基础上改进而来，其强抗碰撞性于 2005 年被王小云院士团队攻破
SHA-256	256	SHA-2 系列的一个版本，由 NIST 设计，用到了 8 个哈希初值以及 32 个哈希常量。该算法被用于生成比特币地址
SHA-512	512	SHA-2 系列的一个版本，由 NIST 设计，用到了 8 个哈希初值和 64 个哈希常量
SHA-3	任意长度	在 2005 年 SHA-1 的强抗碰撞性被攻破的背景下，NIST 开始设计下一代哈希函数 SHA-3。2012 年 10 月，NIST 选择 Keccak 算法作为 SHA-3 的标准算法



散列函數的應用: 比特幣

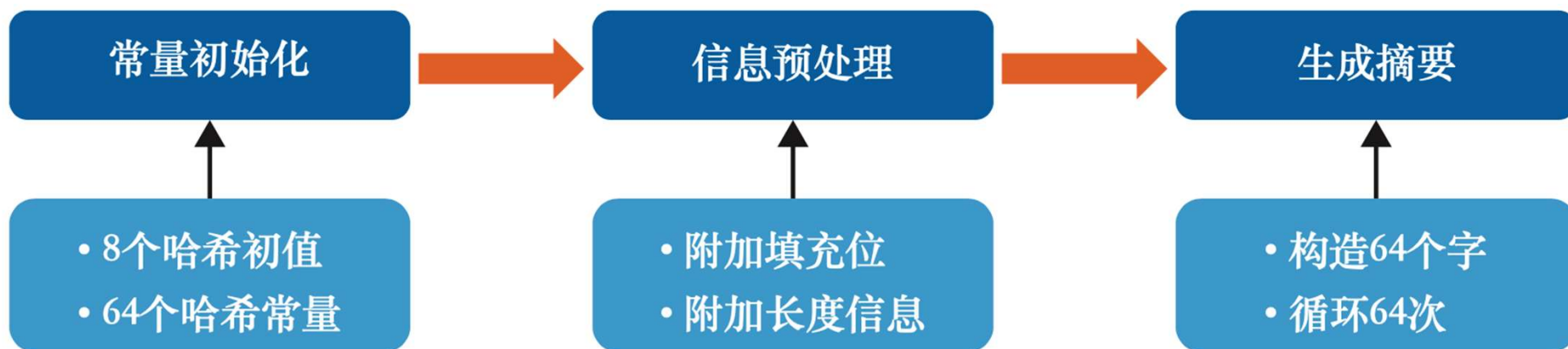


- 比特幣是一種P2P形式的虛擬的加密數字貨幣，使用了SHA256、RIPEMD160
- SHA256
 - 輸出：256位的哈希值
 - 用途：區塊的頭部信息、交易數據、工作量證明、比特幣地址等
- RIPEMD160
 - 輸出：160位的哈希值
 - 用途：比特幣地址生成，可以讓地址更短



SHA256算法

SHA256是SHA-2標準下細分出的一種散列函數。算法的實現主要分為三個步驟：實現可以分為三個部分：常量初始化，信息預處理，生成摘要





SHA256算法

1. 常量初始化

SHA256算法中用到了8個哈希初值以及64個哈希常量。其中8個哈希初值，分別是最開始的連續8個素數（即2,3,5,7,11,13,17,19）的平方根的二進制表示的小數部分的前32位

$h0 := 0x6a09e667$	$h1 := 0xbb67ae85$	$h2 := 0x3c6ef372$	$h3 := 0xa54ff53a$
$h4 := 0x510e527f$	$h5 := 0x9b05688c$	$h6 := 0x1f83d9ab$	$h7 := 0x5be0cd19$

還要初始化64個值，分別是最開始的連續64個素數（從2到311）的立方根的二進制表示的小數部分的前32位

428a2f98	71374491	b5c0fbcf	e9b5dba5	3956c25b	59f111f1	923f82a4	ab1c5ed5
d807aa98	12835b01	243185be	550c7dc3	72be5d74	80deb1fe	9bdc06a7	c19bf174
e49b69c1	efbe4786	0fc19dc6	240ca1cc	2de92c6f	4a7484aa	5cb0a9dc	76f988da
983e5152	a831c66d	b00327c8	bf597fc7	c6e00bf3	d5a79147	06ca6351	14292967
27b70a85	2e1b2138	4d2c6dfc	53380d13	650a7354	766a0abb	81c2c92e	92722c85
a2bfe8a1	a81a664b	c24b8b70	c76c51a3	d192e819	d6990624	f40e3585	106aa070
19a4c116	1e376c08	2748774c	34b0bcb5	391c0cb3	4ed8aa4a	5b9cca4f	682e6ff3
748f82ee	78a5636f	84c87814	8cc70208	90befffa	a4506ceb	bef9a3f7	c67178f2



SHA256算法

2. 信息預處理

信息的預處理分為兩個步驟：附加**填充比特**和**附加長度**。以信息“abc”為例進行預處理演示

- 填充比特：第一個填充的比特為1，其後的填充的比特都為0，直到長度滿足對512取模後餘數是448
- 附加長度：將原始數據的長度信息補到已經進行了填充操作的消息之後

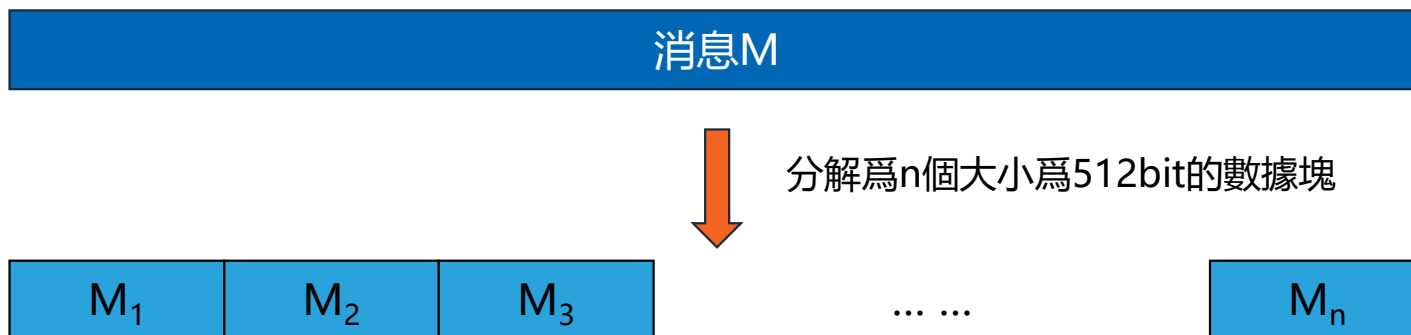
信息	a	b	c
ASCII碼（二進制）	01100001	01100010	01100011
補位（第一位補“1”）	01100001 01100010 01100011 1		
補位（其後位補“0”）	01100001 01100010 01100011 10000000 00000000 ... 00000000 (423個0)		
比特填充結果（十六進制）	61626380 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000		
附加長度結果（十六進制）	61626380 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000018		



SHA256算法

3. 生成摘要

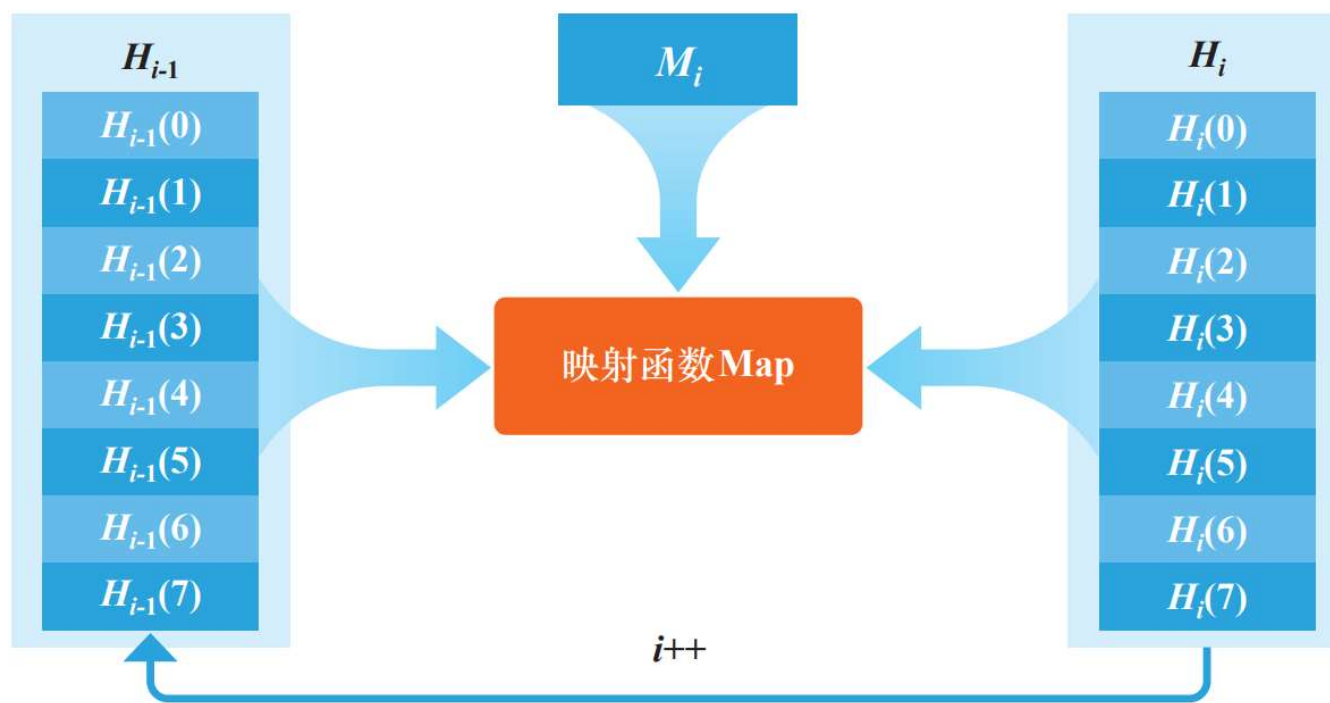
假設消息M可以被分解為n個塊，于是整個算法需要做的就是完成n次迭代，n次迭代的結果就是最終的哈希值，即256bit的數字摘要





SHA256算法

- 一個256-bit的摘要的初始值 H_0 ，經過第一個數據塊進行運算，得到 H_1 ，即完成了第一次迭代， H_1 經過第二個數據塊得到 H_2 ，依次處理，最後得到 H_n ， H_n 即為最終的256-bit消息摘要
- 將每次迭代進行的映射用 $Map(H_{i-1}) = H_i$ 表示，于是迭代可以更形象的展示為：



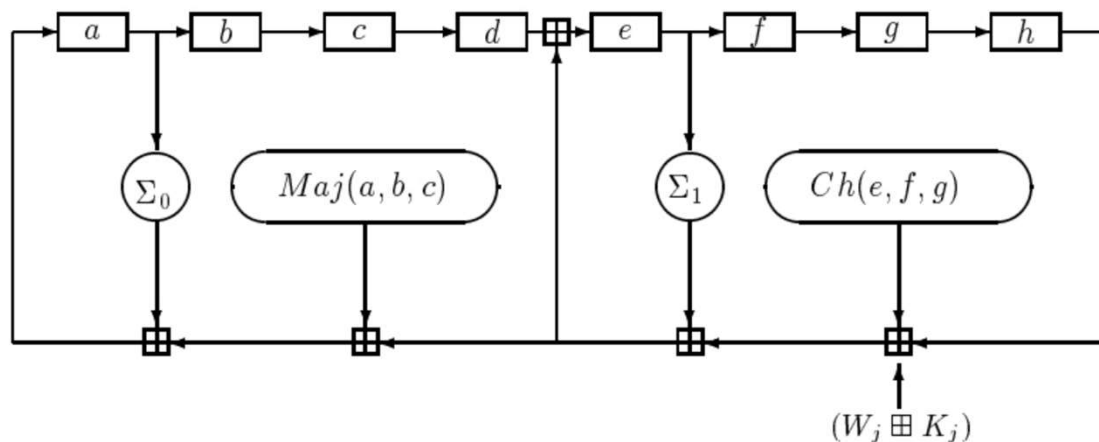


SHA256中的映射函數

Step1: 構造64個32bit字 (word) , 將每一塊分解為16個32-bit的big-endian的字, 記為 $w[0], \dots, w[15]$, 前16個字直接由消息的第 i 個塊分解得到, 其餘的字由如下迭代公式得到:

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}$$

Step2: 進行64次循環



$$Ch(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z)$$

$$Maj(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$$

$$\Sigma_0(x) = S^2(x) \oplus S^{13}(x) \oplus S^{22}(x)$$

$$\Sigma_1(x) = S^6(x) \oplus S^{11}(x) \oplus S^{25}(x)$$

$$\sigma_0(x) = S^7(x) \oplus S^{18}(x) \oplus R^3(x)$$

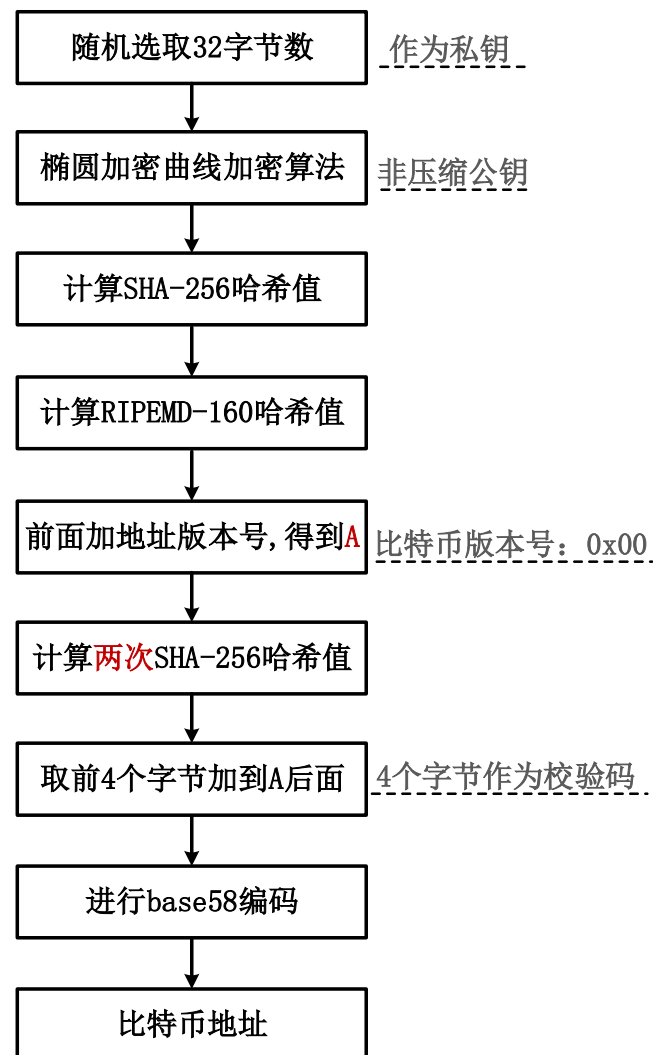
$$\sigma_1(x) = S^{17}(x) \oplus S^{19}(x) \oplus R^{10}(x)$$

$\oplus$	bitwise XOR
$\wedge$	bitwise AND
$\vee$	bitwise OR
$\neg$	bitwise complement
$+$	mod 2^{32} addition
R^n	right shift by n bits
S^n	right rotation by n bits



比特幣地址的生成

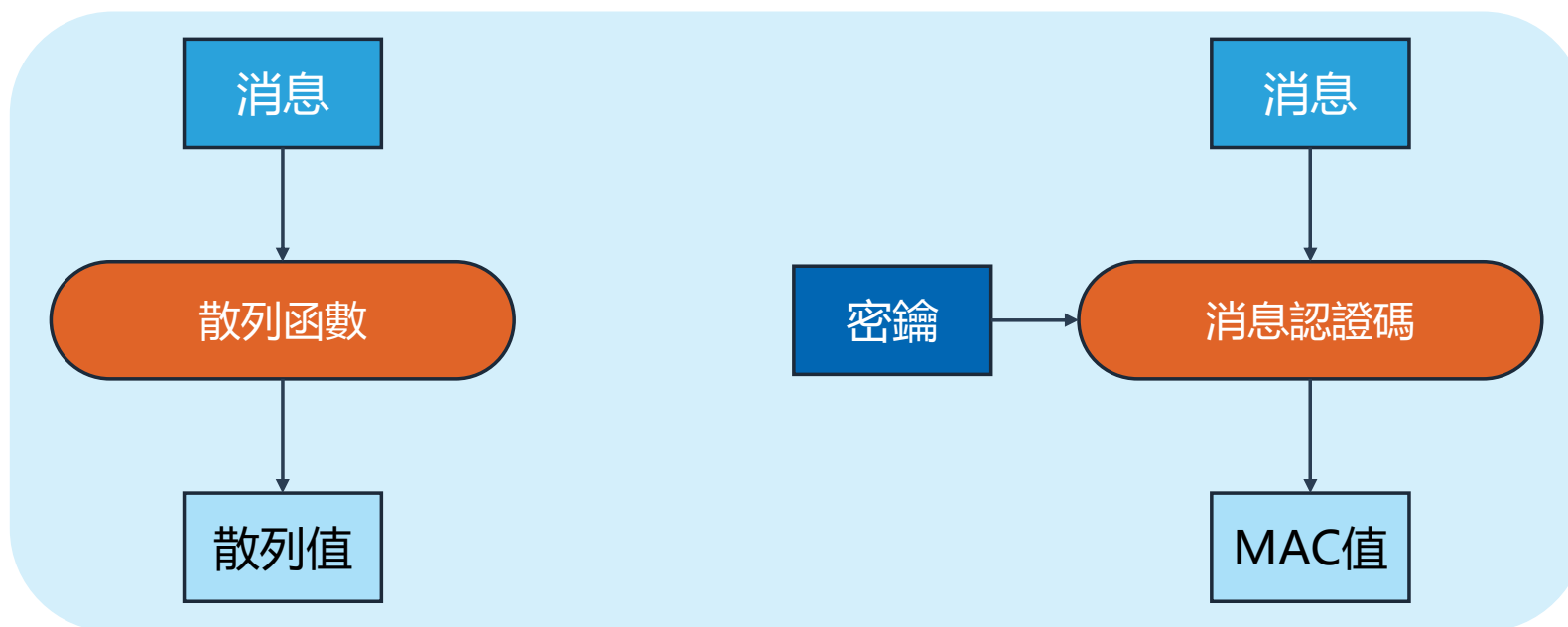
- 比特幣地址的生成：
 - 隨機選取32字節的數作為私鑰
 - 橢圓曲綫加密算法得到非壓縮公鑰
 - 依次計算SHA-256、RIPEMD-160值
 - 上一結果前加地址版本號 (0x00)
 - 計算兩次SHA-256
 - 取前4個字節加到第4步結果後面
 - 進行base58編碼
 - 獲得比特幣地址





消息認證碼

消息認證碼（Message Authentication Code）的輸入包括任意長度的消息和一個發送者與接受者之間共享的密鑰，它可以輸出固定長度的數據，這個數據稱為**MAC值**

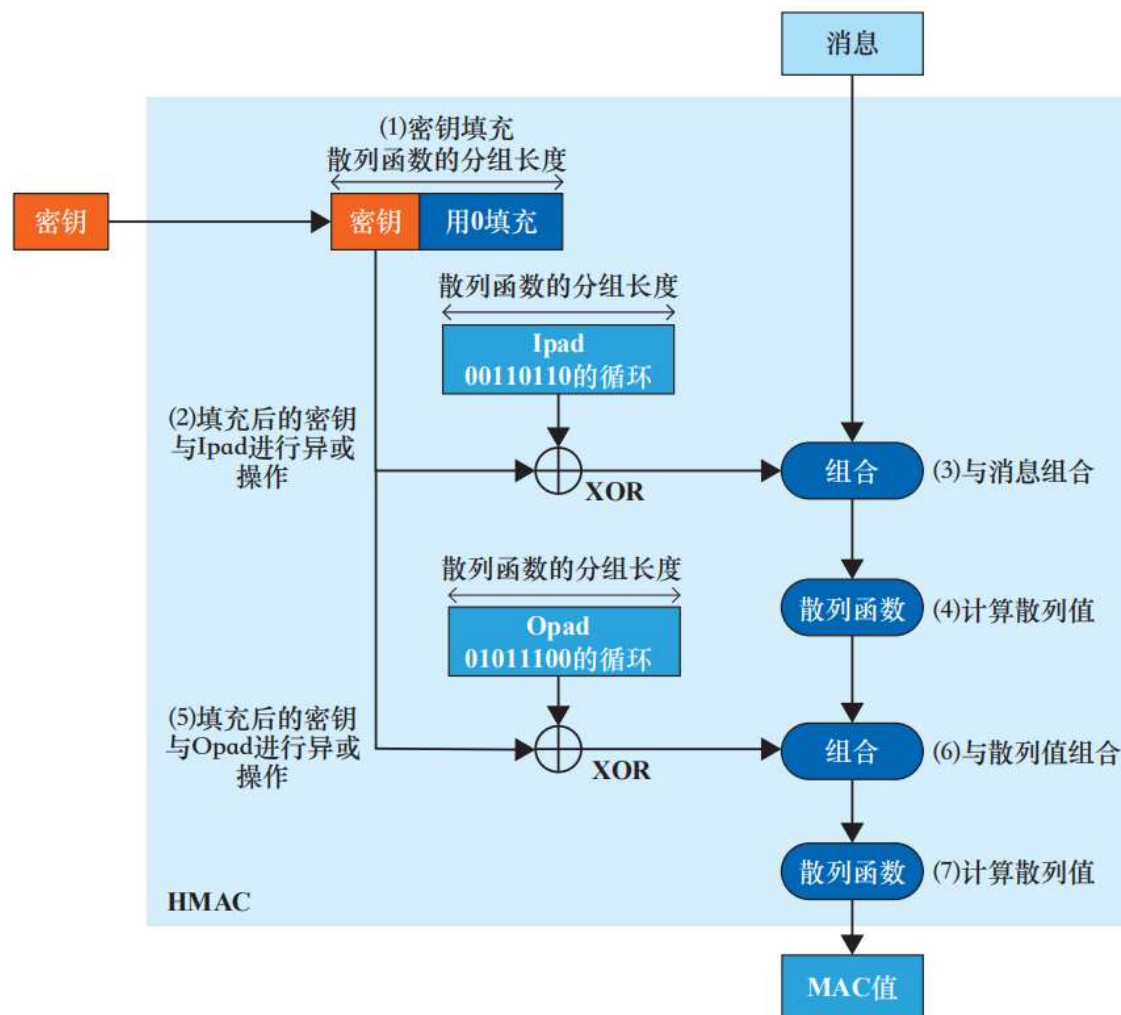


散列函數與消息認證碼的比較



HMAC

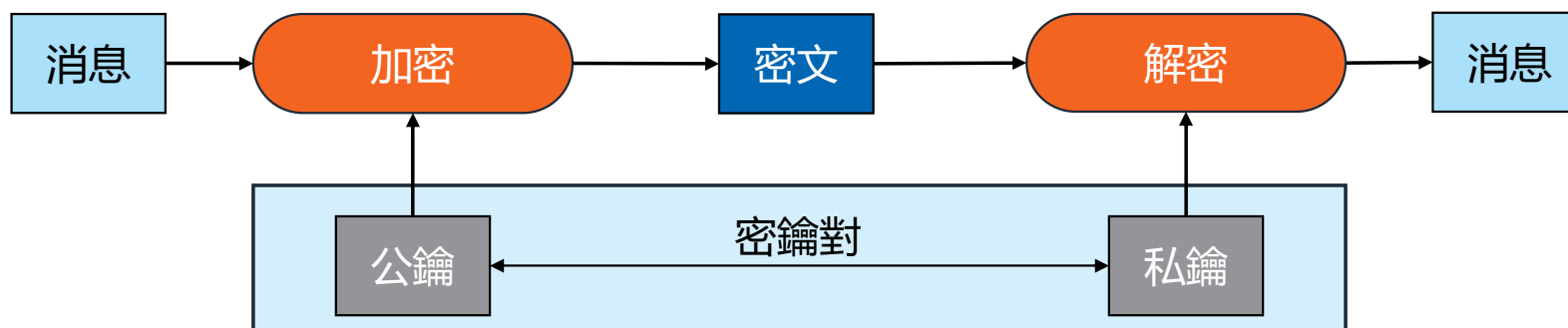
- HMAC是一種使用散列函數來構造消息認證碼的方法
- 該方法所使用的散列函數不僅限于一種，任何高强度的散列函數都可以被用在HMAC
- 使用SHA-1, SHA-256, 所構造的HMAC分別稱為HMAC-SHA1, HMAC-SHA256



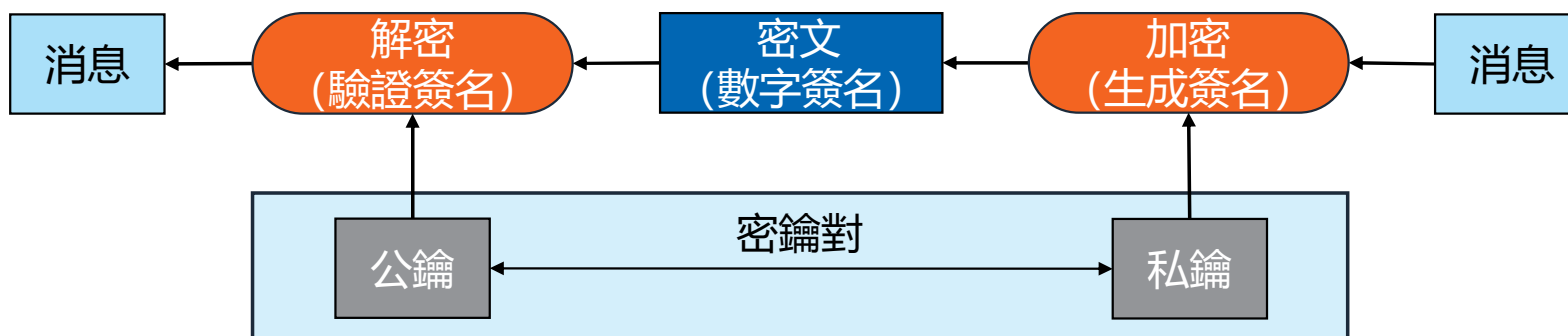


公鑰密碼與數字簽名

(1) 公鑰密碼



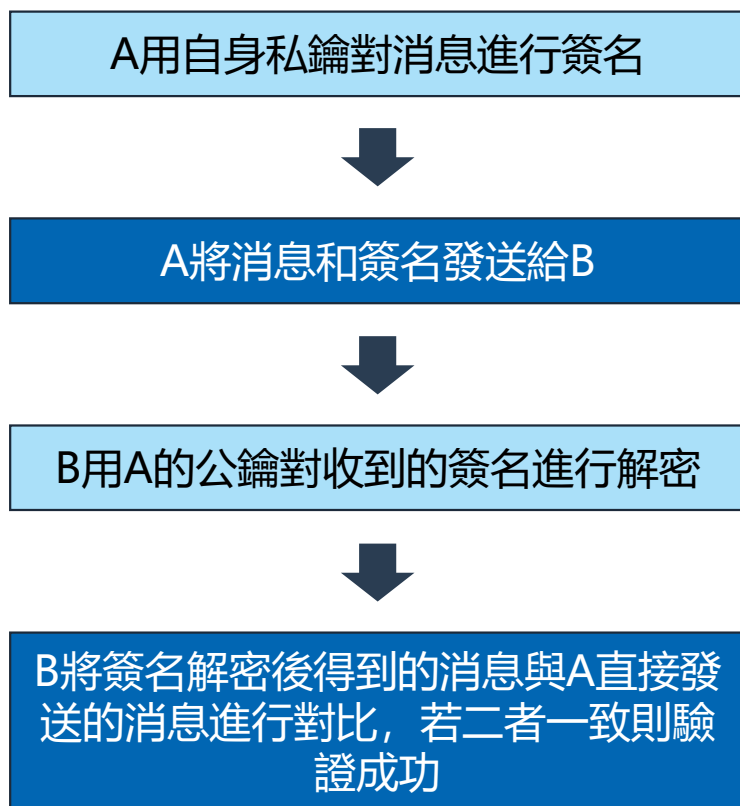
(2) 數字簽名



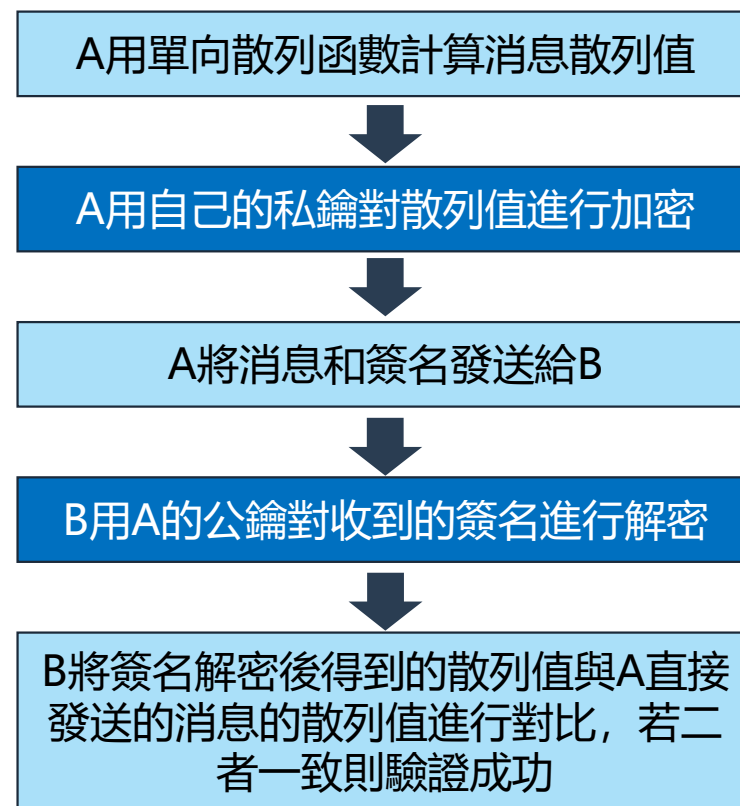


數字簽名的方法

- 直接對消息簽名的方法



- 對消息的散列值簽名的方法





RSA數字簽名方案

數字簽名方案包括3個過程：系統的初始化、產生簽名和驗證簽名

系統的初始化過程

選擇兩個保密的大素數 p 和 q ，計算 $n = p \times q$ ， $l = \text{lcm}(p-1, q-1)$ ，選擇整數 e ，滿足 $1 < e < l$ ，且 $\text{gcd}(e, l) = 1$ ；計算 $d \times e = 1 \bmod l$ ；以 $\{e, n\}$ 為公鑰， d 為私鑰

簽名產生過程

設消息為 M ，對其簽名為 $S \equiv M^d \bmod n$ ，並將 (M, S) 發送給簽名驗證者

簽名驗證過程

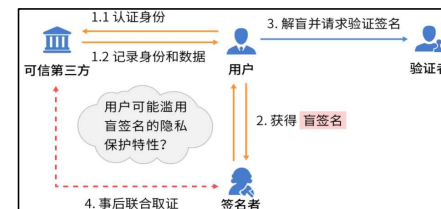
接收方在收到消息 M 和簽名 S 後，驗證 $M \equiv S^e \bmod n$ 是否成立。若成立，則發送方的簽名有效；若不成立，則簽名無效



數字簽名的分類

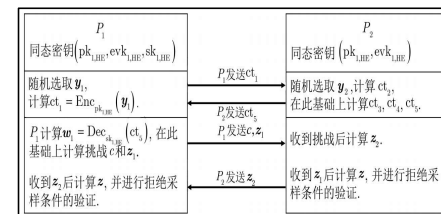
盲簽名

簽名者不知道代簽名文件內容時使用的數字簽名



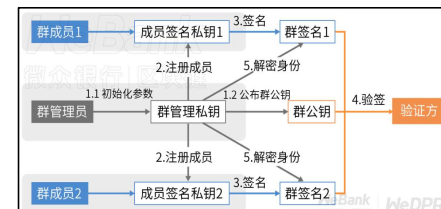
門限簽名

如果一個群體中有n個人，那麼至少需要p個人簽名才視為有效簽名($n > p$)



群簽名

一個群體由多個成員組成，某個成員可以代表整個群體來進行數字簽名，而且該成員作為簽名者可以被驗證



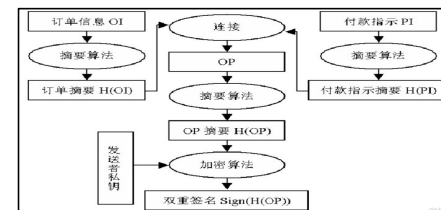
代理簽名

密鑰的所有者可以將簽名權利授予第三方，獲得權力的第三方可以進行數字簽名



雙重簽名

簽名者希望有個中間人在他與驗證者之間進行驗證授權操作





數字簽名的應用

1. 網站認證

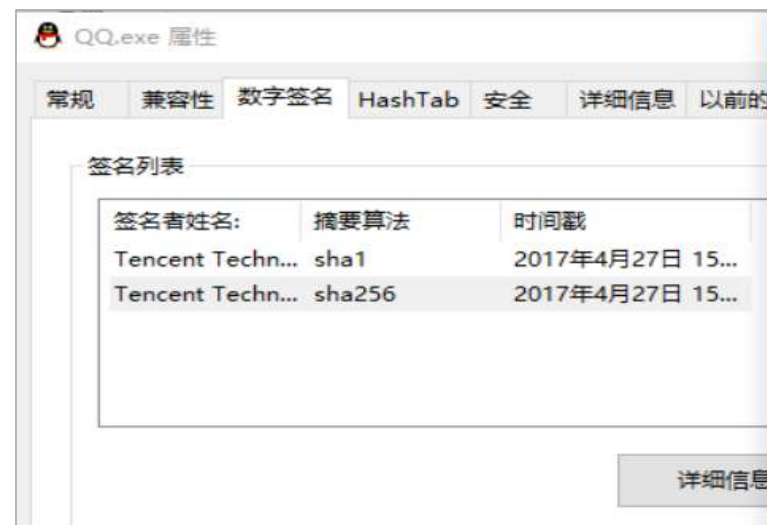
通過對網站域名信息、主體身份信息、域名權屬信息等進行嚴格鑒證審核，並利用PKI數字簽名技術形成不可篡改的認證標識，在互聯網終端以安全可靠的方式進行展示，使網民更直觀確認網站的真實身份

2. 比特幣

比特幣是一種完全匿名的數字貨幣，它的身份認證是基於ECDSA，比特幣的賬戶地址就是對公鑰計算摘要得到的，並用私鑰確認賬戶擁有者

3. 代碼簽名

如果Windows上的可執行程序程序來源於正規公司，通常它會有代碼簽名，用於確保其來源可靠且未被篡改



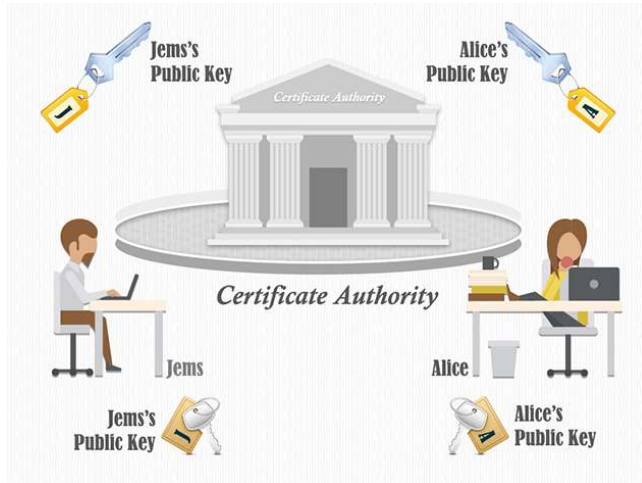


第5節 公鑰基礎設施PKI

- ✓ 體系結構
- ✓ 信任模型
- ✓ 安全問題
- ✓ 應用場景



公鑰基礎設施PKI (Public Key Infrastructure)



CA (Certificate Authority)：可信賴的第三方權威機構，負責給網絡實體頒發身份證明書，將用戶所持有的公開密鑰與其身份信息結合在一起

PKI (Public Key Infrastructure)：由CA, 數字證書等組成的公鑰與身份驗證體系

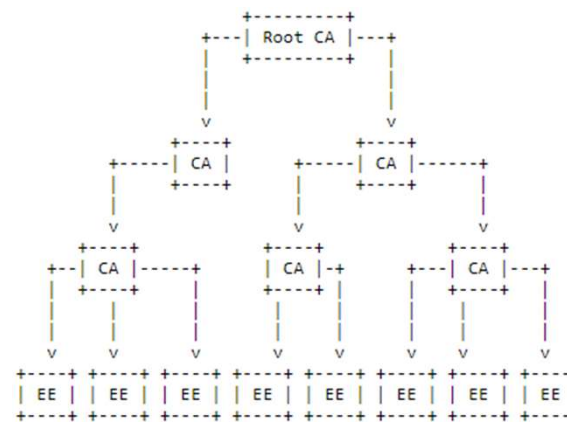


Figure 1 - Sample Hierarchical PKI

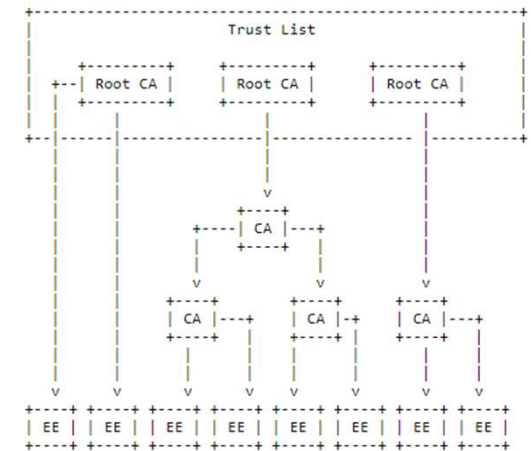
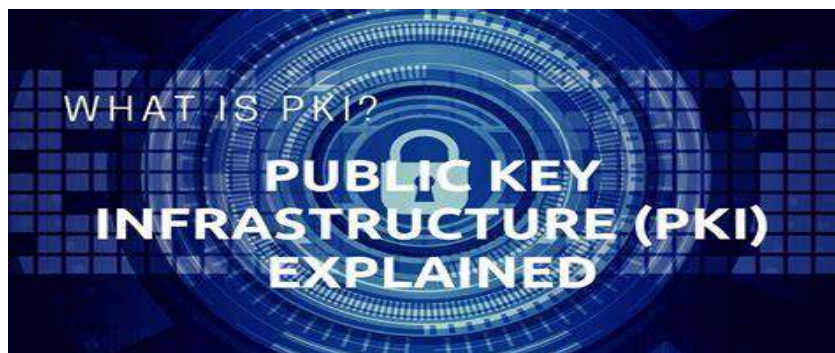


Figure 2 - Multi-Rooted Hierarchical PKI



PKI的作用

- 公鑰基礎設施 (Public Key Infrastructure) 以公鑰技術為基礎，提供具有普適性安全服務的基礎設施
- PKI 旨在從技術上解決網上身份認證、信息的完整性和不可抵賴性等安全問題



例如：

1. 在WEB服務器和瀏覽器之間的通信
2. 電子郵件
3. 電子數據內部交換
4. 在Internet上的信用卡交易
5. 虛擬專用網VPN





保證公鑰的真實性

- 公鑰的分發雖然不需要保密，但需要保證公鑰的真實性
- 就好像銀行的客服電話，雖然不需要保密，但需要保證真實性



- 數字證書和公鑰基礎設施PKI就是爲了實現在公鑰分發過程中確保公鑰真實性

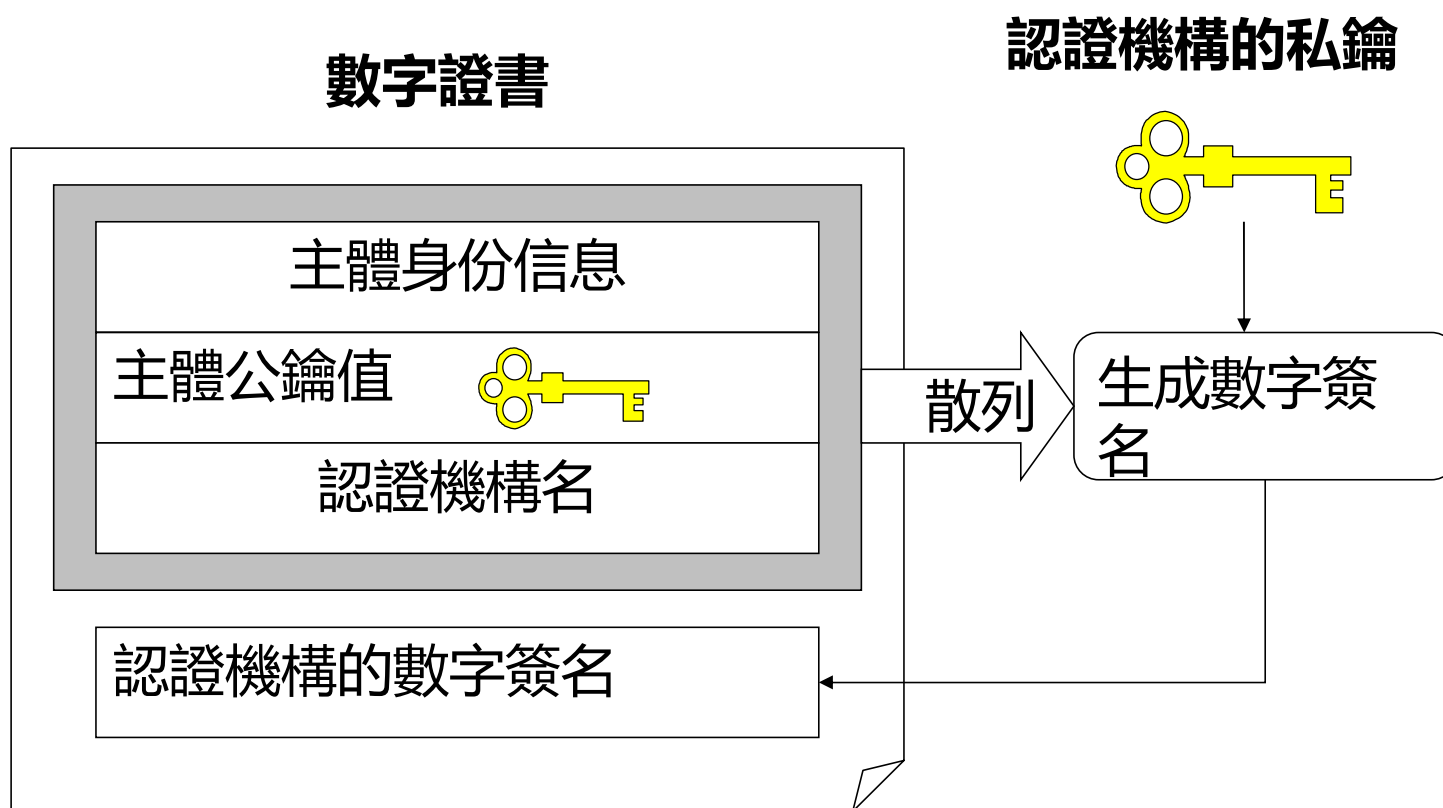


公鑰基礎設施PKI的核心組件：數字證書



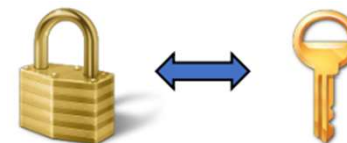


公鑰基礎設施PKI的核心組件：數字證書



注意：

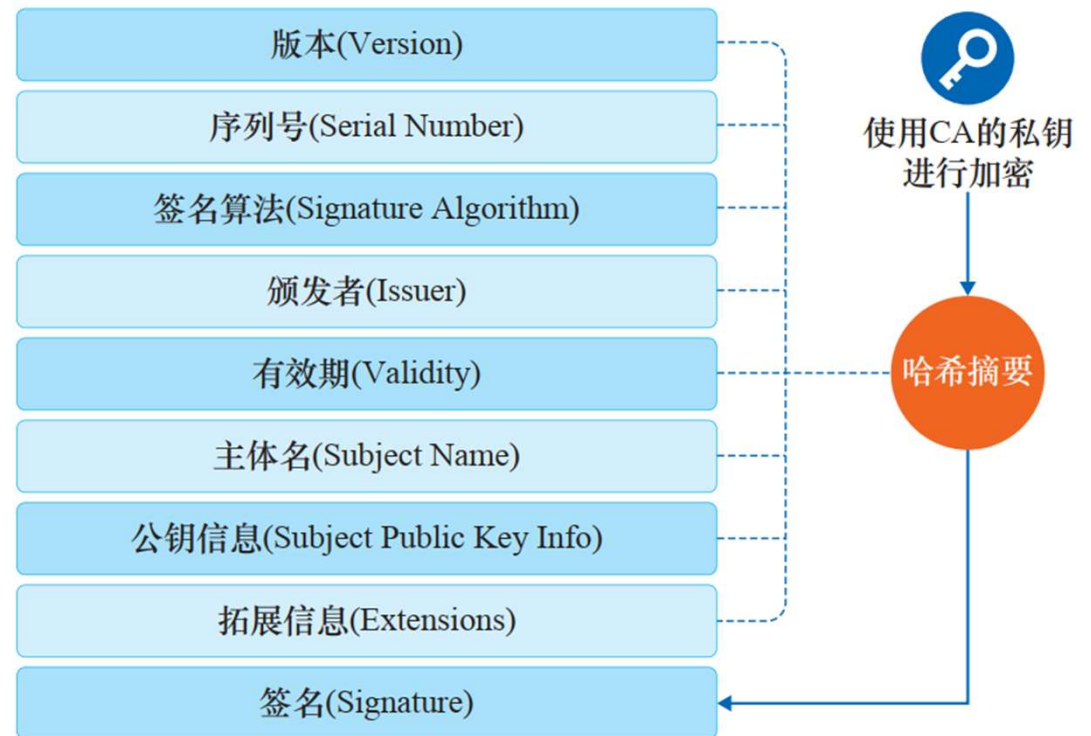
- 雖然證書只包含了公鑰，但必須與對應私鑰配合使用
- 數字證書和私鑰的關係有點像鎖和鑰匙的關係。雖然鎖裏面沒有包含鑰匙，但是鎖必須和鑰匙配合使用





X.509證書格式

- 現實中有各種各樣的數字證書，如PGP、SET、IPSec
- 目前應用最廣泛的證書格式是國際電信聯盟ITU提出的X.509版本
- X.509于1988年頒布，1993和1995年兩次修改
- IETF針對X.509在Internet環境的應用，頒布了RFC2459





X.509證書實例

CFCA是經中國人民銀行和國家
信息安全管理機構批准成立的
國家級權威的安全認證機構，
是重要的國家金融信息安全基
礎設施之一



CFCA EV ROOT



CFCA EV ROOT

根证书颁发机构

过期时间：2029年12月31日 星期一 中国标准时间 上午11:07:01

此证书有效

信任

细节

主题名称

国家/地区 CN

组织 China Financial Certification Authority

常用名称 CFCA EV ROOT

签发者名称

国家/地区 CN

组织 China Financial Certification Authority

常用名称 CFCA EV ROOT

序列号 407555286

版本 3

签名算法 带 RSA 加密的 SHA-256 (1.2.840.113549.1.1.11)

参数 无

在此之前无效 2012年8月8日 星期三 中国标准时间 上午11:07:01

在此之后无效 2029年12月31日 星期一 中国标准时间 上午11:07:01

公共密钥信息

算法 RSA 加密 (1.2.840.113549.1.1.1)

参数 无

公共密钥 512 字节: D7 5D 6B CD 10 3F 1F 05 ...

指数 65537

密钥大小 4,096 位

密钥使用 验证



PKI 基本功能



用戶注册

證書申請

密鑰產生

密鑰更新

密鑰備份

密鑰恢復

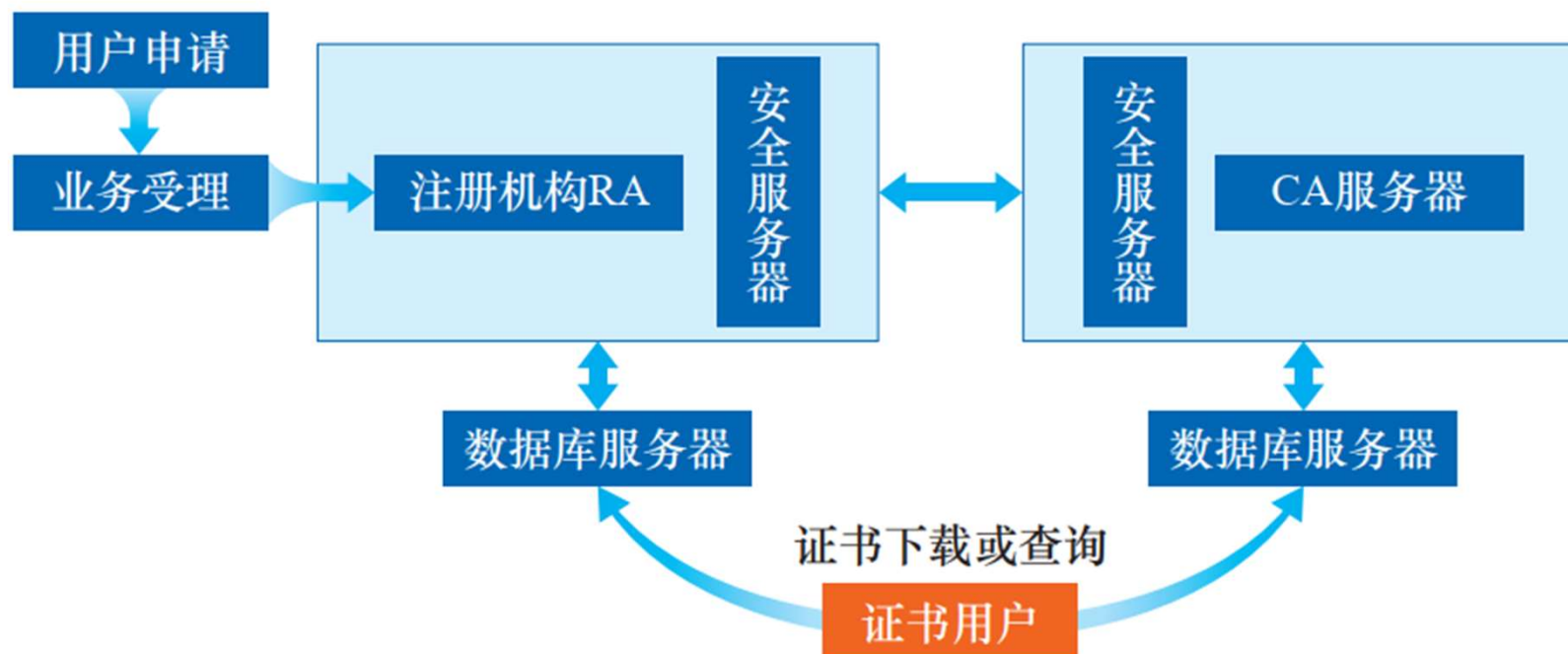
證書作廢

證書歸檔



PKI 體系結構

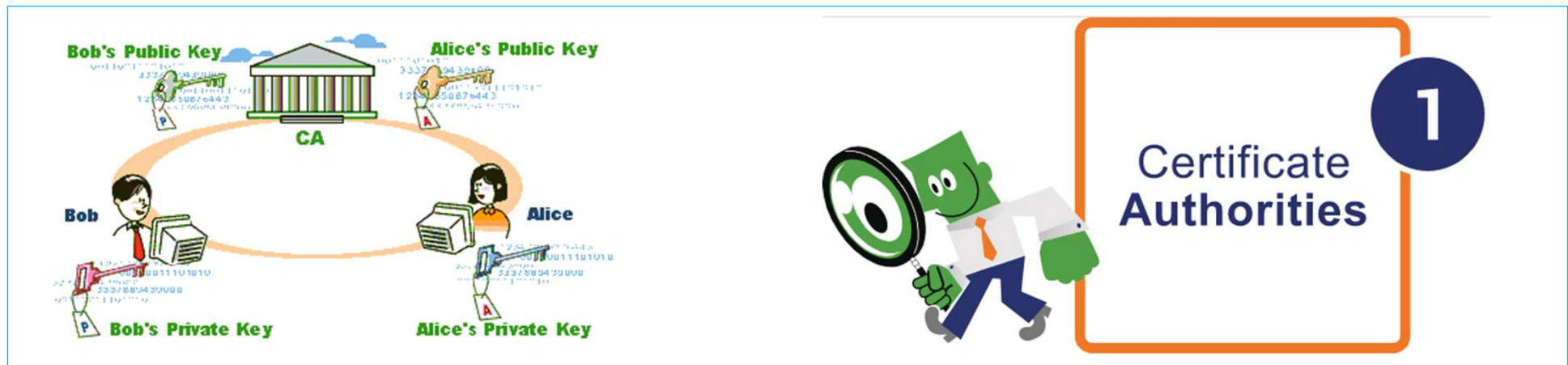
PKI體系通常由終端實體用戶、證書認證機構(CA)、證書注册機構(RA)和證書數據庫，以及安全服務器組成





CA (Certification Authority)

- 證書機制是目前被廣泛採用的一種安全機制，使用證書機制的前提是建立認證中心(CA)以及配套的注冊中心(RA)系統
- 證書依靠一個可靠的第三方機構驗證，CA專門提供這種服務





CA的職能

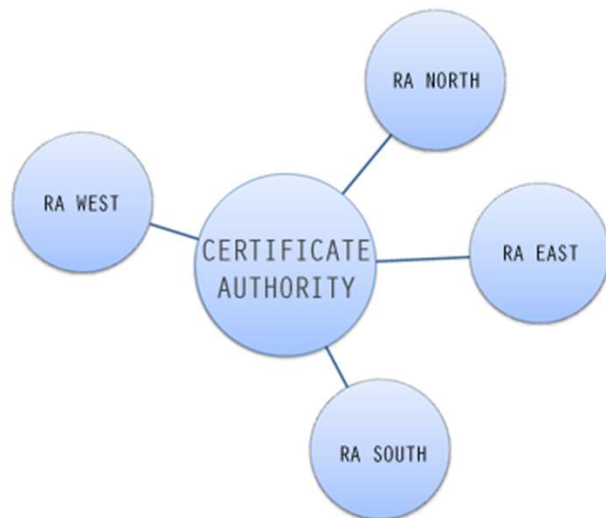
- 產生自身證書并傳輸給安全服務器
- 驗證用戶的身份，產生、分配并管理PKI結構下的所有用戶的證書
- 核心功能就是發放和管理數字證書
- 負責用戶證書的黑名單登記和發布





RA (Registration Authority)

注册中心RA系統負責證書申請者的信息錄入、審核以及證書發放等工作；
對發放的證書完成相應管理功能

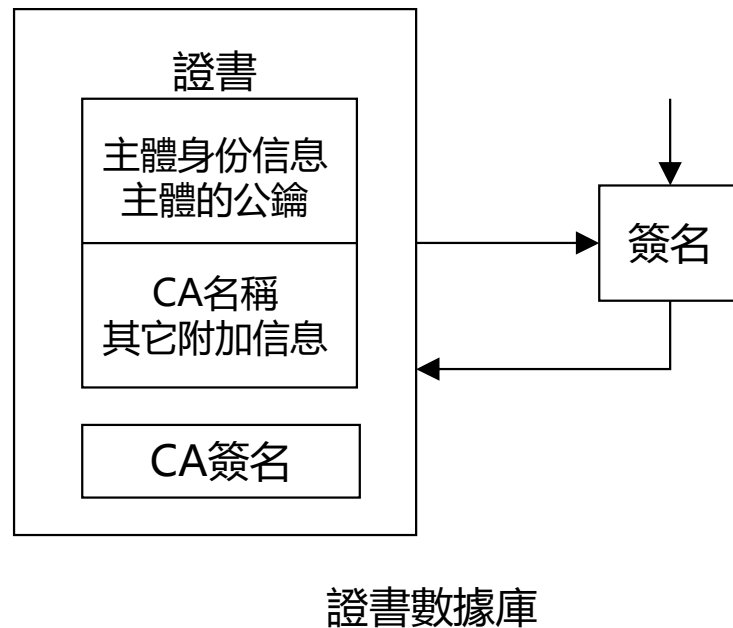


What is a Registration Authority?

"The RA ensures that the public key is bound to the individual to which it is assigned in a way that ensures non-repudiation"



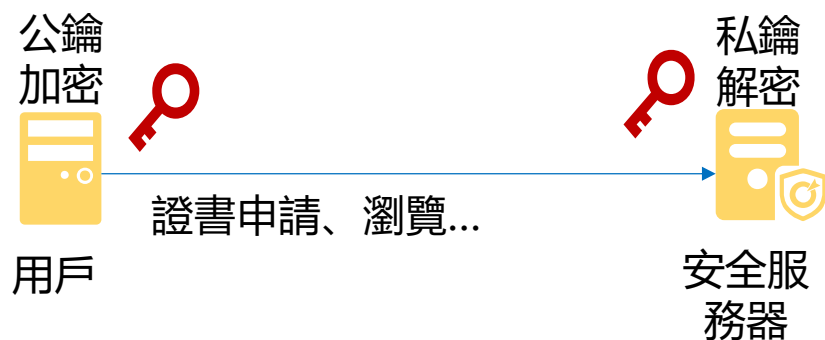
證書數據庫



- 證書集中存放地，用戶可以從此處獲得其他用戶的證書和公鑰
- 證書數據庫服務器用于認證機構中數據（如密鑰和用戶信息等）、日志和統計信息的存儲和管理



安全服務器



- 安全服務器面向普通用戶，提供證書申請、瀏覽、撤銷列表及證書下載等安全服務
- 用戶首先得到安全服務器證書（CA頒發），然後用戶與服務器之間的所有通信均以安全服務器的密鑰進行加密傳輸，通過安全服務器的私鑰解密得到明文，保證了證書申請和傳輸過程中的信息安全性



PKI 信任模型

PKI信任模型：證書用戶、證書主體、各CA間的證書認證關係稱為PKI信任模型

➡ 用戶的信任起點在何處

01

01

各國都建立自己的PKI，內部建立不同行業地區PKI

➡ 信任在系統中如何被傳遞

02

02

不同的PKI之間的互聯互通和相互信任

解決兩個問題

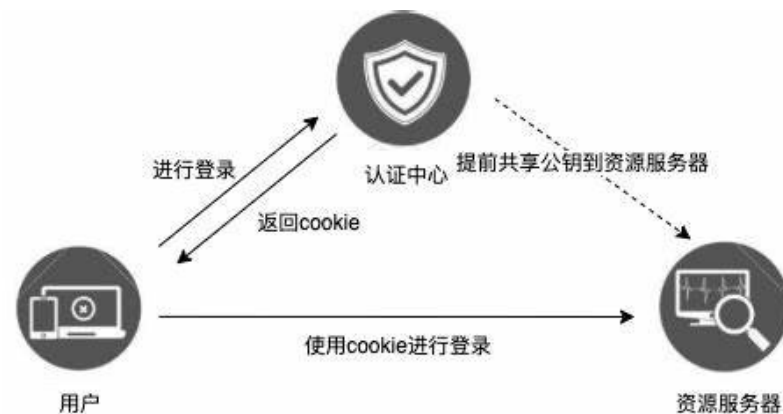
建立形式



PKI 信任模型

信任模型提供了建立和管理信任關係的框架，
按照有無第三方可信機構參與，信任可劃分為**直接信任**和**第三方信任**

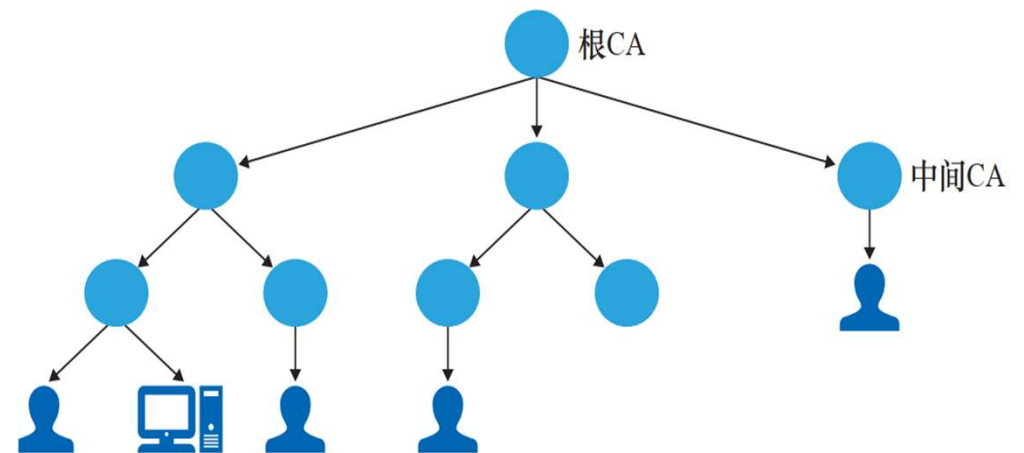
第三方信任是指兩個實體以前沒有建立起信任關係，第三方為兩者的可信任性進行了擔保，是目前網絡安全中普遍采用的信任模式





CA的層次結構

- 對於大型權威機構，簽發證書的工作不能僅僅由一個CA來完成，可建立一個CA層次結構
- 一個CA一般為一個安全域（security domain）的有限群體發放證書
- 每個CA覆蓋一定作用範圍，不同用戶群體往往擁有不同的CA

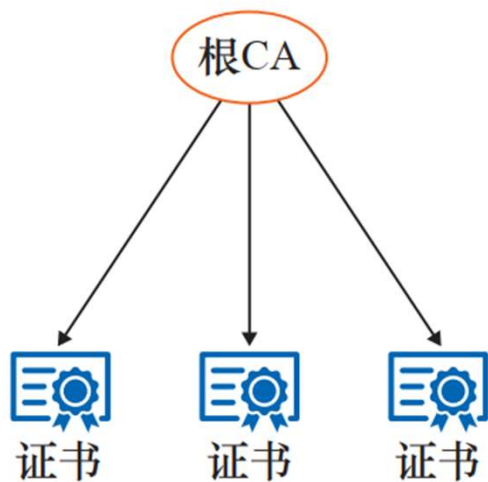




CA的信任模型

A

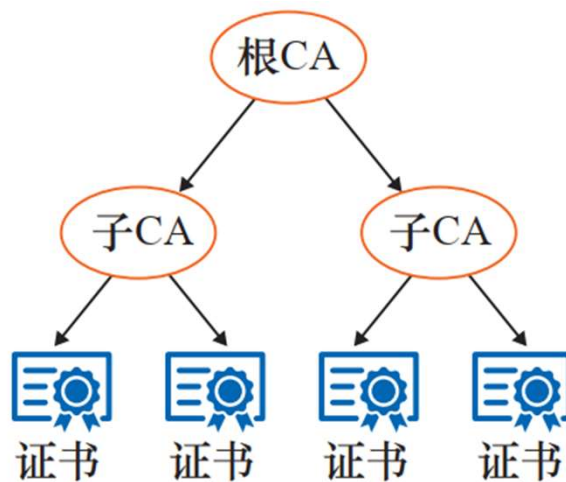
單CA信任模型：
基本信任模型



(a) 单CA信任模型

B

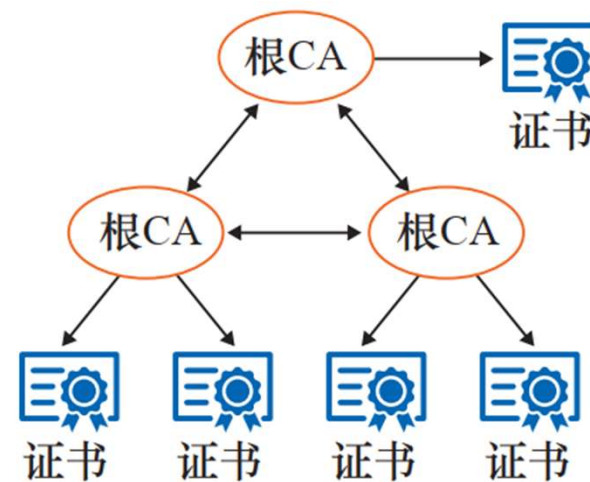
層次信任模型：
主、從CA建立分級結構



(b) 层次信任模型

C

分布式信任模型：
CA間存在交叉認證



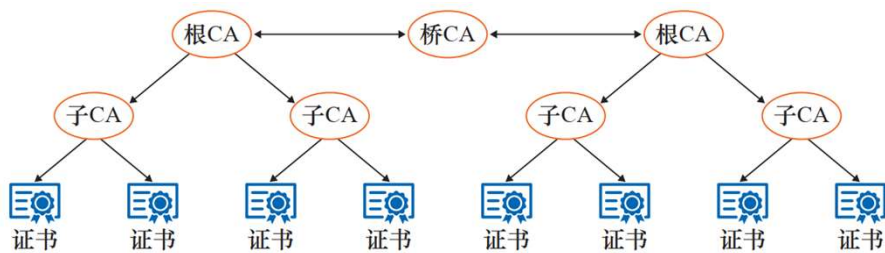
(c) 分布式信任模型



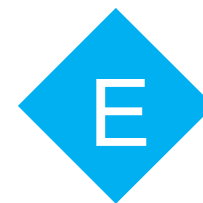
CA的信任模型



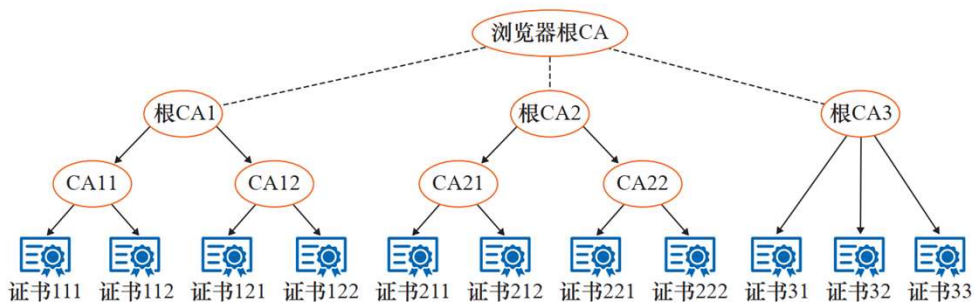
橋CA信任模型：
中心輻射式信任模型



(D) 橋CA信任模型



WEB信任模型：
構建在Web瀏覽器基礎上

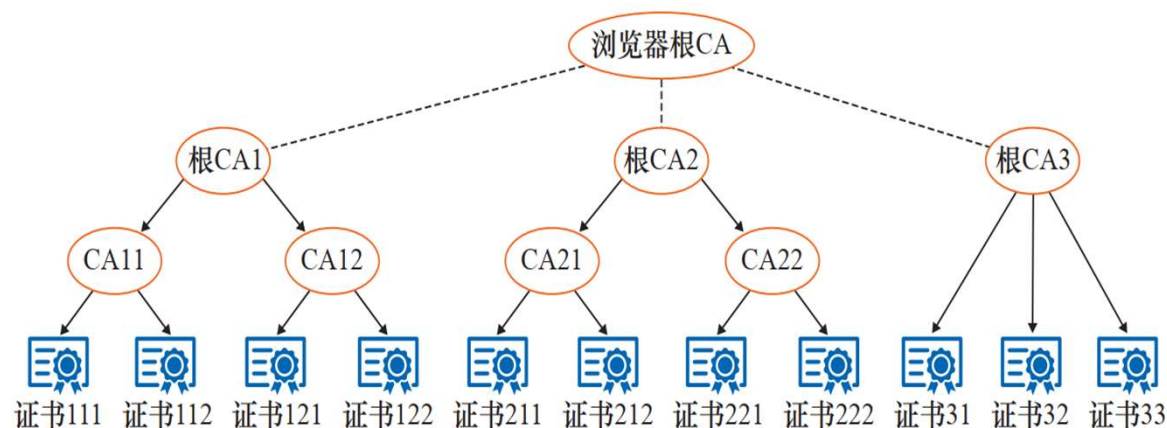


(E) WEB信任模型



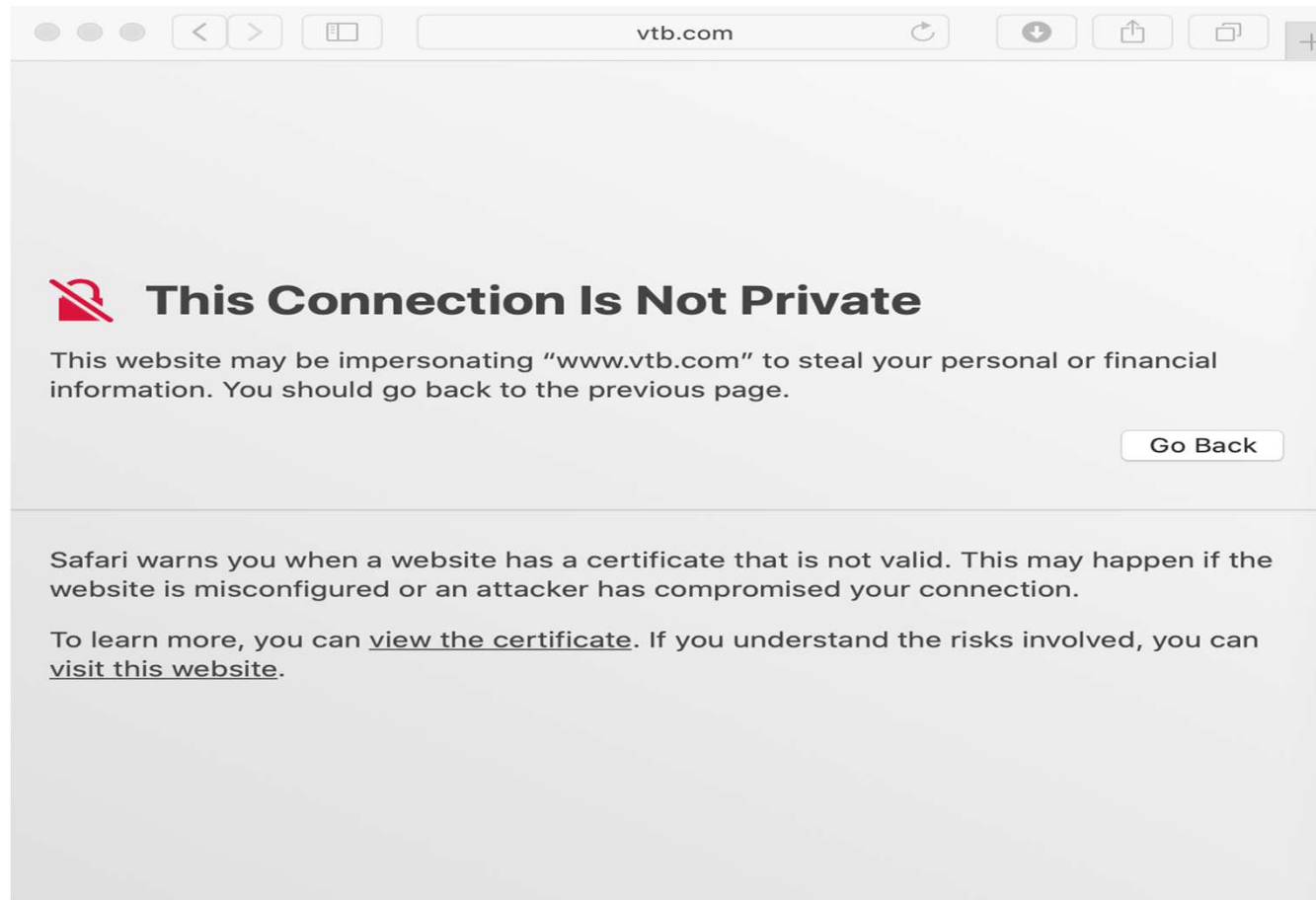
Web 信任模型

- Web信任模型構建在Web瀏覽器的基礎上，瀏覽器中內置多個根CA
- 根CA間是平行的，瀏覽器用戶同時信任多個根CA并把這些根CA作為自己的信任錨





Web 信任實例：俄羅斯外貿銀行證書 (1)





Web 信任實例：俄羅斯外貿銀行證書 (2)

crt.sh

Certificate Search

Criteria

ID = '5828347935'

crt.sh ID	5828347935																																				
Summary	Leaf certificate																																				
Certificate Transparency	Log entries for this certificate: <table><thead><tr><th>Timestamp</th><th>Entry #</th><th>Log Operator</th><th>Log URL</th></tr></thead><tbody><tr><td>2021-12-19 19:56:49 UTC</td><td>568562107</td><td>Sectigo</td><td>https://mammoth.ct.comodo.com</td></tr><tr><td>2021-12-19 19:56:49 UTC</td><td>409342372</td><td>Google</td><td>https://ct.googleapis.com/logs/argon2022</td></tr><tr><td>2021-12-19 19:56:50 UTC</td><td>171813954</td><td>Let's Encrypt</td><td>https://oak.ct.letsencrypt.org/2022</td></tr><tr><td>2021-12-20 05:17:41 UTC</td><td>530136563</td><td>Google</td><td>https://ct.googleapis.com/logs/xenon2022</td></tr></tbody></table>	Timestamp	Entry #	Log Operator	Log URL	2021-12-19 19:56:49 UTC	568562107	Sectigo	https://mammoth.ct.comodo.com	2021-12-19 19:56:49 UTC	409342372	Google	https://ct.googleapis.com/logs/argon2022	2021-12-19 19:56:50 UTC	171813954	Let's Encrypt	https://oak.ct.letsencrypt.org/2022	2021-12-20 05:17:41 UTC	530136563	Google	https://ct.googleapis.com/logs/xenon2022																
Timestamp	Entry #	Log Operator	Log URL																																		
2021-12-19 19:56:49 UTC	568562107	Sectigo	https://mammoth.ct.comodo.com																																		
2021-12-19 19:56:49 UTC	409342372	Google	https://ct.googleapis.com/logs/argon2022																																		
2021-12-19 19:56:50 UTC	171813954	Let's Encrypt	https://oak.ct.letsencrypt.org/2022																																		
2021-12-20 05:17:41 UTC	530136563	Google	https://ct.googleapis.com/logs/xenon2022																																		
Revocation	Report a problem with this certificate to the CA <table><thead><tr><th>Mechanism</th><th>Provider</th><th>Status</th><th>Revocation Date</th><th>Last Observed in CRL</th><th>Last Checked (Error)</th></tr></thead><tbody><tr><td>OCSP</td><td>The CA</td><td>Check</td><td>?</td><td>n/a</td><td>?</td></tr><tr><td>CRL</td><td>The CA</td><td>Revoked</td><td>2022-03-01 00:14:47 UTC</td><td>2022-03-25 09:41:08 UTC</td><td>2022-03-25 13:41:15 UTC</td></tr><tr><td>CRLSet/Blocklist</td><td>Google</td><td>Not Revoked</td><td>n/a</td><td>n/a</td><td>n/a</td></tr><tr><td>disallowedcert.stl</td><td>Microsoft</td><td>Not Revoked</td><td>n/a</td><td>n/a</td><td>n/a</td></tr><tr><td>OneCRL</td><td>Mozilla</td><td>Not Revoked</td><td>n/a</td><td>n/a</td><td>n/a</td></tr></tbody></table>	Mechanism	Provider	Status	Revocation Date	Last Observed in CRL	Last Checked (Error)	OCSP	The CA	Check	?	n/a	?	CRL	The CA	Revoked	2022-03-01 00:14:47 UTC	2022-03-25 09:41:08 UTC	2022-03-25 13:41:15 UTC	CRLSet/Blocklist	Google	Not Revoked	n/a	n/a	n/a	disallowedcert.stl	Microsoft	Not Revoked	n/a	n/a	n/a	OneCRL	Mozilla	Not Revoked	n/a	n/a	n/a
Mechanism	Provider	Status	Revocation Date	Last Observed in CRL	Last Checked (Error)																																
OCSP	The CA	Check	?	n/a	?																																
CRL	The CA	Revoked	2022-03-01 00:14:47 UTC	2022-03-25 09:41:08 UTC	2022-03-25 13:41:15 UTC																																
CRLSet/Blocklist	Google	Not Revoked	n/a	n/a	n/a																																
disallowedcert.stl	Microsoft	Not Revoked	n/a	n/a	n/a																																
OneCRL	Mozilla	Not Revoked	n/a	n/a	n/a																																
Certificate Fingerprints	SHA-256 F529EEABC121D950B72208FF6FD71E4A844CC5EBA8D594A135CA66C1DCC1B0E0 SHA-1 314DAEA50ADD8DA4289367B4653FCCF7D0BEAD0F																																				
Certificate ASN.1 Graph pv	Certificate: Data: Version: 3 (0x2) Serial Number: 06:1c:b6:86:54:d6:6c:8b:dd:61:bb:ce:ce:7a:8e:85 Signature Algorithm: sha256WithRSAEncryption Issuer: (CA ID: 62131) commonName = Thawte RSA CA 2018 organizationalUnitName = www.digicert.com organizationName = DigiCert Inc countryName = US Validity Not Before: Oct 7 00:00:00 2021 GMT Not After : Oct 7 23:59:59 2022 GMT Subject: commonName = online.vtb.ru organizationName = VTB Bank (PJSC) localityName = Санкт-Петербург countryName = RU Subject Public Key Info: Public Key Algorithm: rsaEncryption RSA Public-Key: (2048 bit)																																				



Web 信任實例：俄羅斯外貿銀行證書 (3)

The screenshot shows a web browser window with the VTB Bank website. The address bar displays `vtb.com/o-b`. A certificate details window is open, showing the following information:

- GlobalSign**
 - VTB BANK GCC R6 OV CA 2021
 - *online.vtb.ru
- 主题名称**
 - 国家或地区 RU
 - 省/市/自治区 Saint Petersburg
 - 所在地 Saint Petersburg
 - 组织 VTB Bank (PJSC)
 - 常用名称 *.online.vtb.ru
- 签发者名称**
 - 国家或地区 RU
 - 组织 VTB BANK (PJSC)
 - 常用名称 VTB BANK GCC R6 OV CA 2021
- 序列号** 75 DB 48 7A 9C C1 9F AA 3F 52 37 29
- 版本** 3
- 签名算法** 带RSA加密的SHA-384 (1.2.840.113549.1.1.12)
- 参数** 无
- 在此之前无效** 2022年3月1日 星期二 中国标准时间 17:06:08
- 在此之后无效** 2023年4月2日 星期日 中国标准时间 17:06:07
- 公共密钥信息**
 - 算法 RSA加密 (1.2.840.113549.1.1.1)
 - 参数 无
 - 公共密钥 256 字节: AB FC EF AC 1B 4B 17 E1 ...
 - 指数 65537
 - 密钥大小 2,048 位
 - 密钥使用 加密, 验证, 封装, 派生
 - 签名 512 字节: C9 3E 2D A8 A1 EB D1 2D ...
- 扩展** 密钥使用 (2.5.29.15)
- 关键** 是
- 用途** 数字签名, 密钥加密

The browser window also shows the VTB Bank logo and the text "About VTB Group" and "VTB Group is a global provider of credit institutions and financial the financial markets."



Web 信任模型

實現方式

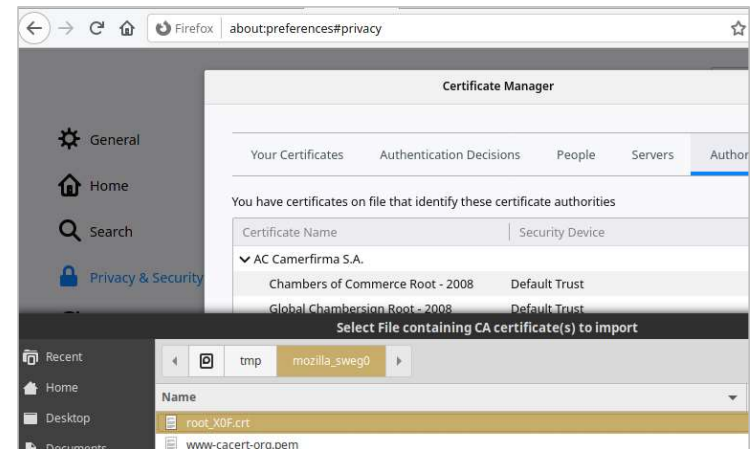
Web信任模型通過與相關域進行互連而不是擴大現有的主體群來使用戶實體成為在瀏覽器中所給出的所有域的依托方

模型特點

具有分布式信任結構模型的特點，但根本上更類似于認證機構的嚴格層次結構模型

認證形式

瀏覽器廠商起到了根CA的作用，而與被嵌入的密鑰相對應的CA就是它所認證的CA



Digital Certificate



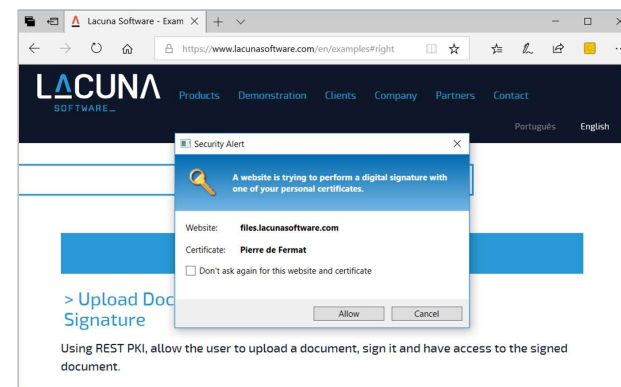


Web 信任模型的安全隱患

01

預裝公鑰信任問題

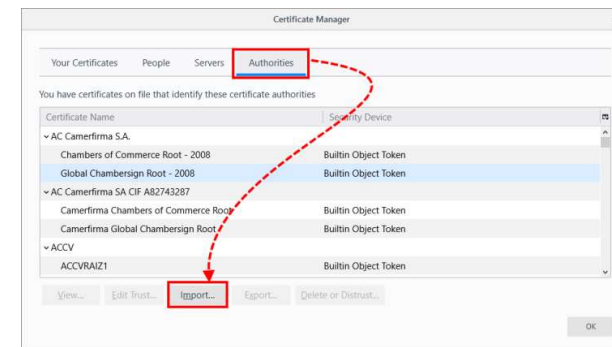
用戶自動地信任預裝公鑰，即使這些 CA 中有一個是從沒有認真核實被認證的實體，這時安全性將被完全破壞



02

缺乏根密鑰撤銷機制

沒有實用的機制來撤銷嵌入到瀏覽器中的根密鑰

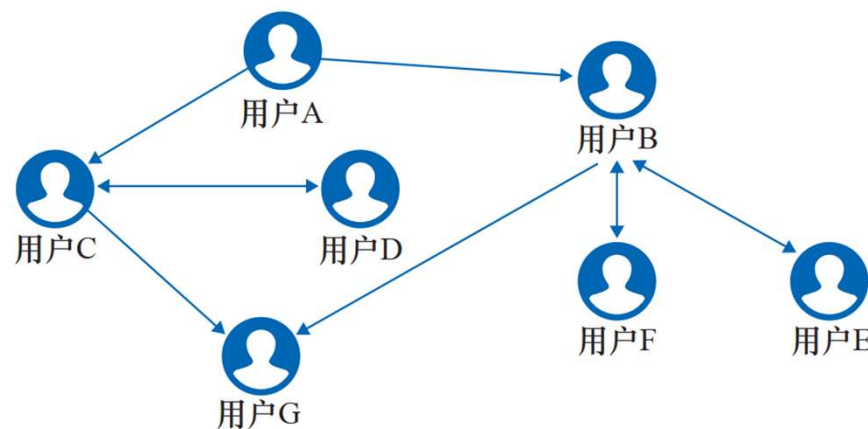




以用戶為中心的信任模型： PGP

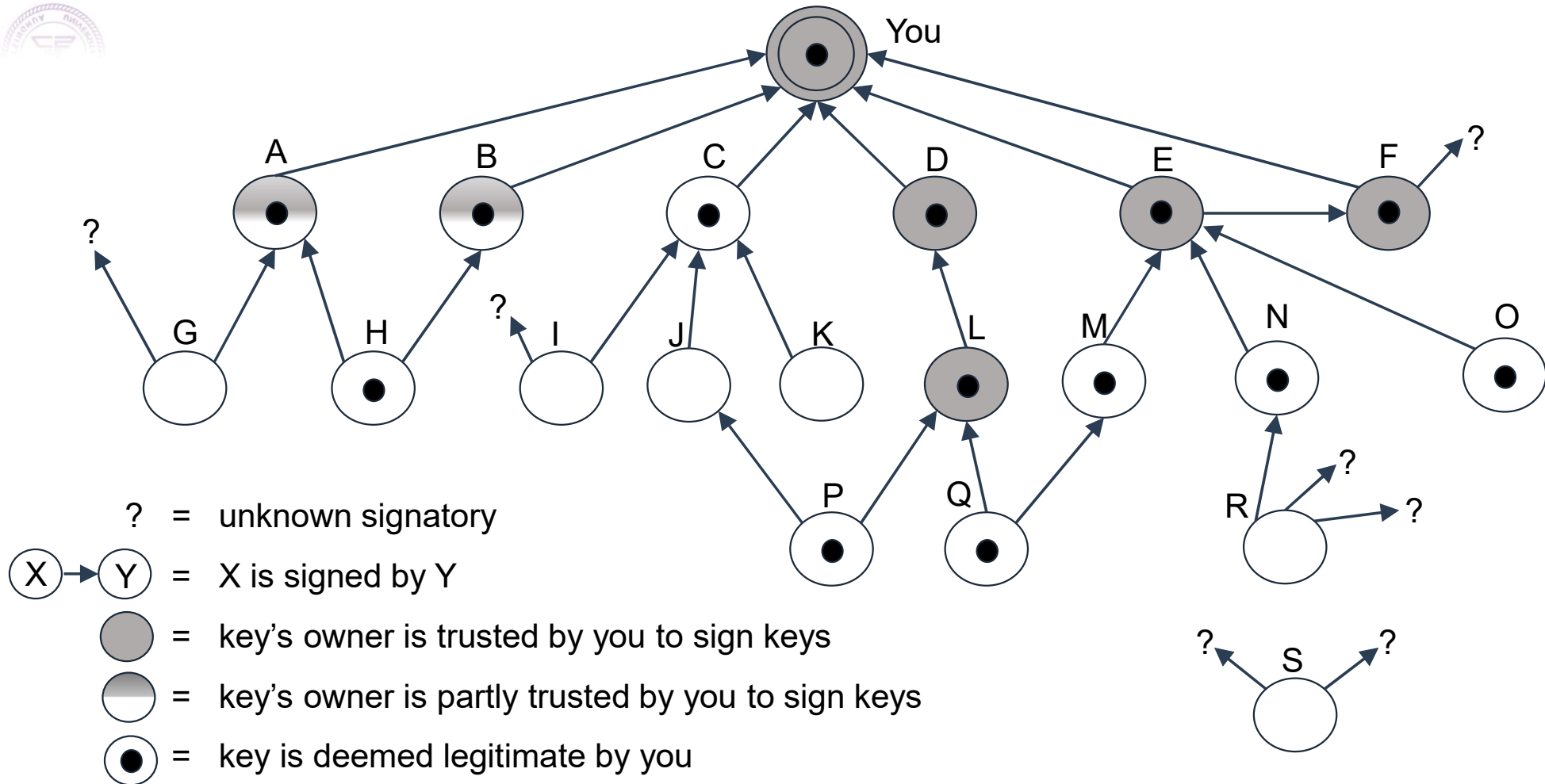
由每個用戶自己決定信任哪些證書

- Pretty Good Privacy (PGP) 中，用戶通過擔當 CA（簽署其他實體的公鑰）和使其公鑰被其他人認證建立“信任網”（Web of Trust）
- 當 A 收到一個屬 B 的證書時，發現這個證書由不認識的 D 簽署的，但是 D 的證書是由 A 認識且信任的 C 簽署，于是 A 決定信任 B 的密鑰，也可決定不接受 B 的密鑰





以用戶為中心的信任模型：PGP



PGP實例，用戶決定是否采信



第7節 總結和展望





數據生命周期與密碼技術

密碼技術在數據生命周期中可以發揮哪些作用？

根據數據分級分類的要求，
采用基于密碼技術的數據
標籤，確保數據生命周期
中處理過程可溯可查

采用密碼技術解決存儲安
全性問題；利用公鑰密碼
技術或哈希函數驗證遠程
數據的完整性

采用非對稱公私鑰對構建
可信身份；利用數字簽名
技術進行用戶身份認證，
實現數據確權

數據采集

數據傳輸

數據存儲

數據處理

數據交換

數據銷毀

數字簽名、數據摘要等技
術可防止數據在傳輸過程
中經由骨幹網絡節點轉發
時被未經授權的查看、篡
改或破壞

采用全同態加密等技術解
決利用雲計算過程中導致
的數據所有權和管理權分
離所帶來的數據安全問題

對雲存儲中的數據進行多
次加密，當需要銷毀數據
時，可直接刪除密鑰，提
高數據銷毀效率



總結

圍繞數據加密的主題，介紹了保障網絡空間安全的核心技術——密碼學，縱觀密碼學的發展歷程，闡述了密碼學的理論基礎，總結了密碼學的應用現狀



第一節 密碼學簡史

- 古典密碼
- 近代密碼
- 現代密碼

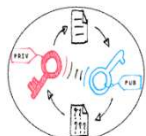
回顧歷史，瞭解密碼學演進方向



第二節 對稱密碼

- 分組密碼
- DES算法
- 流密碼

總結對稱密碼的加密解密原理



第三節 公鑰密碼

- 密鑰分發問題
- 公鑰密碼
- RSA算法
- 應用場景

剖析公鑰密碼的來龍去脉



第四節 摘要與簽名

- 散列函數
- 消息認證碼
- 數字簽名

介紹校驗數據完整性的工具方法



第五節 公鑰基礎設施PKI

- 體系結構
- 信任模型
- 安全問題
- 應用場景

明確PKI的意義及如何構建安全的PKI



第六節 密碼分析技術

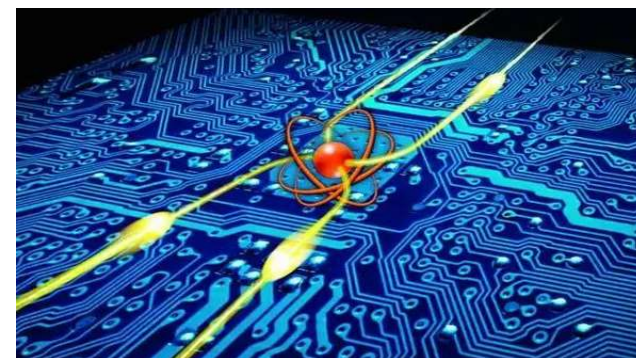
- 唯密文攻擊
- 已知明文攻擊
- 選擇明文攻擊
- 選擇密文攻擊
- 選擇密鑰攻擊

梳理密碼分析領域的技術體系



展望

密碼技術發展為解決數據安全問題提出新的技術思路



- **基于編碼的密碼體制、基于格的密碼體制、多變量密碼體制等後量子密碼體制**突飛猛進，為數據安全技術發展注入新動力
- **抵抗量子攻擊的密碼體制**受到關注



展望

突破受制于人的密碼技術，構建內生安全體系，保障國家數據安全



- 開展**自主可控密碼體制**開發設計，以及軟硬件基礎設施、網絡加密協議、應用等由美國主導控制下的**極限對抗研究**
- 構建覆蓋**人工智能、區塊鏈、5G、量子通信、雲計算**等新型信息技術的**數據加密體制**